JMORF/MORS — Morpho-Syntax

The Structure of the Lexicon

Francis Bond
Palacký University
https://fcbond.github.io/
bond@ieee.org

Lecture 7
Location: KB-1.11

Overview

- Lexical types: the lexeme hierarchy, lexical terminology and default inheritance
- Activity: all the constraints on ***give***
- Lexical rules: inflectional, derivational and post-inflectional
- Activity: the word structure for ***writers***
- Problem: arguments in Japanese

Lexical Types

Overview

- Motivation for lexical hierarchy
- Lexical terminology
- Default inheritance
- Tour of the lexeme hierarchy
- The Case Constraint
- *pos* vs. *lexeme*
- In-class activity: all the constraints on ***give***

Motivation for the Lexical Hierarchy

- We've streamlined our grammar rules...
  - ...by stating some constraints as general principles
  - ...and locating lots of information in the lexicon.

- Our lexical entries currently stipulate a lot of information that is common across many entries and should be stated only once. Examples?

Motivation for the Lexical Hierarchy

- We've streamlined our grammar rules...
  - ...by stating some constraints as general principles
  - ...and locating lots of information in the lexicon.

- Our lexical entries currently stipulate a lot of information that is common across many entries and should be stated only once. Examples?
  - Every common noun: $\left[\text{HEAD} \begin{bmatrix} \textit{noun} \\ \text{AGR} \ \left[\text{PER} \ \textit{3rd}\right] \end{bmatrix}\right]$, $\left[\text{SPR} \ \langle \text{DP} \rangle\right]$, $\left[\text{MODE} \ \textit{ref}\right]$
  - Every verb: $\left[\text{HEAD} \ \textit{verb}\right]$, $\left[\text{MODE} \ \textit{prop}\right]$, an NP subject first on ARG-ST
  - Every noun and verb: the SHAC

The Ideal Lexical Entry

- Ideally, particular lexical entries need only give phonological form, the semantic contribution, and any constraints truly idiosyncratic to the lexical entry.
  - We don't get quite that far in this book, because of our 'cutesy' role names (LOVER, GIVER, ...): the entry still has to say which argument plays which role
  - Implemented grammars such as the ERG use generic role labels (ARG1, ARG2), so the mapping from arguments to roles can live in the lexical type

Lexemes and Words (in a General Linguistic Sense)

- **Lexeme**: An abstract proto-word which gives rise to genuine words. We refer to lexemes by their 'dictionary form', e.g. 'the lexeme *run*' or 'the lexeme *dog*'.

- **Word**: A particular pairing of form and meaning. *Running* and *ran* are different words

Lexical Entries, Lexical Types & Lexical Rules

- **Lexical entries** capture the similarities among ***run, runs, running,*** and ***ran***.

- The **lexical type hierarchy** captures the similarities among ***run, sleep,*** and ***laugh***, among those and other verbs like ***devour*** and ***hand***, and among those and other words like ***book***.

  Q What do ***devour*** and ***book*** have in common?

- **Lexical rules** capture the similarities among ***runs, sleeps, devours, hands,***...

Lexical Entries, Lexical Types & Lexical Rules

- **Lexical entries** capture the similarities among ***run, runs, running,*** and ***ran***.

- The **lexical type hierarchy** captures the similarities among ***run, sleep,*** and ***laugh***, among those and other verbs like ***devour*** and ***hand***, and among those and other words like ***book***.

  Q What do ***devour*** and ***book*** have in common?
  A The SHAC: each agrees with its specifier

- **Lexical rules** capture the similarities among ***runs, sleeps, devours, hands,***...

Lexical Terminology

The book uses seven related terms

- Lexeme
- Lexical entry
- Lexical sequence
- 'Family of lexical sequences'
- Lexical rule
- Lexical rule instantiation
- Word structure

Lexical Terminology

- **Lexeme**
- **Lexical entry**
- **Lexical sequence**
- **'Family of lexical sequences'**
- Lexical rule
- Lexical rule instantiation
- Word structure

lexeme

- A type in our grammar
- The root of the sub-hierarchy of all the lexeme types we use in defining lexical entries
- So 'a lexeme' loosely refers to the type *lexeme* or one of its subtypes
- These types are generalizations about combinations of constraints
  - e.g. the lexeme type *pn-lxm* encodes the generalization that proper nouns are (usually) singular
  - the lexeme type *stv-lxm* encodes the generalization that a transitive verb takes two arguments

Lexical Entry

- A piece of the grammar that we write in the lexicon
  - e.g. 'look, we have ***book***, it's of type *cntn-lxm*, and we have ***give***, it's of type *dtv-lxm*'

$$\left\langle \text{book} \;,\; \begin{bmatrix} \textit{cntn-lxm} \\ \text{SEM} \; \begin{bmatrix} \text{INDEX} \;\; i \\ \text{RESTR} \; \left\langle \begin{bmatrix} \text{RELN} \;\; \textit{book} \\ \text{INST} \;\; i \end{bmatrix} \right\rangle \end{bmatrix} \end{bmatrix} \right\rangle \qquad \left\langle \text{give} \;,\; \begin{bmatrix} \textit{dtv-lxm} \\ \text{SEM} \; \begin{bmatrix} \text{INDEX} \;\; s \\ \text{RESTR} \; \left\langle \begin{bmatrix} \text{RELN} & \textit{give} \\ \text{SIT} & s \\ \text{GIVER} & i \\ \text{RECIPIENT} & j \\ \text{GIFT} & k \end{bmatrix} \right\rangle \end{bmatrix} \\ \text{ARG-ST} \; \langle \text{X}_i, \; \text{Y}_j, \; \text{Z}_k \rangle \end{bmatrix} \right\rangle$$

Lexical Sequence

- A fully resolved feature structure that is a pair of some orthographic or phonological string with some feature structure of type *lexeme* or *word*

- There is a type *l-sequence* to encode this
  - the INPUT and OUTPUT features of lexical rules have values of type *l-sequence*

TYPE	FEATURES/CONSTRAINTS	IST
list		*feat-struc*
list(τ)	FIRST τ REST *list*(τ)	*list*
l-sequence	FIRST *atom* REST ⟨*word*⟩ | ⟨*lexeme*⟩	*list*

In plain English: an *l-sequence* is a two-element list, a string followed by one *word* or *lexeme*.

Lexical Sequences?

$$\left\langle \text{book} , \begin{bmatrix} \textit{cntn-lxm} \\ \text{SYN} & \begin{bmatrix} \text{HEAD} & \begin{bmatrix} \textit{noun} \\ \text{AGR} \ \boxed{1} & \begin{bmatrix} \textit{3sing} \\ \text{PER} \ \ \textit{3rd} \\ \text{NUM} \ \ \textit{sg} \\ \text{GEND} \ \ \textit{neut} \end{bmatrix} \\ \text{CASE} \ \ \textit{nom} \end{bmatrix} \\ \text{VAL} & \begin{bmatrix} \text{SPR} & \langle [\text{AGR} \ \boxed{1}] \rangle \\ \text{COMPS} & \langle\rangle \\ \text{MOD} & \langle\rangle \end{bmatrix} \end{bmatrix} \\ \text{SEM} & \begin{bmatrix} \text{MODE} \ \ \textit{ref} \\ \text{INDEX} \ \ i \\ \text{RESTR} \ \left\langle \begin{bmatrix} \text{RELN} \ \ \textit{book} \\ \text{INST} \ \ i \end{bmatrix} \right\rangle \end{bmatrix} \\ \text{ARG-ST} & \langle \text{DP}_i [\text{COUNT} \ +] \rangle \end{bmatrix} \right\rangle$$

$$\left\langle \text{book} , \begin{bmatrix} \textit{cntn-lxm} \\ \text{SYN} & \begin{bmatrix} \text{HEAD} & \begin{bmatrix} \textit{noun} \\ \text{AGR} \ \boxed{1} & \begin{bmatrix} \textit{plural} \\ \text{PER} \ \ \textit{3rd} \\ \text{NUM} \ \ \textit{pl} \end{bmatrix} \\ \text{CASE} \ \ \textit{acc} \end{bmatrix} \\ \text{VAL} & \begin{bmatrix} \text{SPR} & \langle [\text{AGR} \ \boxed{1}] \rangle \\ \text{COMPS} & \langle\rangle \\ \text{MOD} & \langle\rangle \end{bmatrix} \end{bmatrix} \\ \text{SEM} & \begin{bmatrix} \text{MODE} \ \ \textit{ref} \\ \text{INDEX} \ \ i \\ \text{RESTR} \ \left\langle \begin{bmatrix} \text{RELN} \ \ \textit{book} \\ \text{INST} \ \ i \end{bmatrix} \right\rangle \end{bmatrix} \\ \text{ARG-ST} & \langle \text{DP}_i [\text{COUNT} \ +] \rangle \end{bmatrix} \right\rangle$$

Q Technically, neither of these is a lexical sequence. Why?

Lexical Sequences?

$$\left\langle \text{book} , \begin{bmatrix} \textit{cntn-lxm} \\ \text{SYN} & \begin{bmatrix} \text{HEAD} & \begin{bmatrix} \textit{noun} \\ \text{AGR} \ \boxed{1} & \begin{bmatrix} \textit{3sing} \\ \text{PER} \ \textit{3rd} \\ \text{NUM} \ \textit{sg} \\ \text{GEND} \ \textit{neut} \end{bmatrix} \\ \text{CASE} \ \textit{nom} \end{bmatrix} \\ \text{VAL} & \begin{bmatrix} \text{SPR} & \langle [\text{AGR} \ \boxed{1}] \rangle \\ \text{COMPS} & \langle \rangle \\ \text{MOD} & \langle \rangle \end{bmatrix} \end{bmatrix} \\ \text{SEM} & \begin{bmatrix} \text{MODE} \ \textit{ref} \\ \text{INDEX} \ i \\ \text{RESTR} \ \left\langle \begin{bmatrix} \text{RELN} \ \textit{book} \\ \text{INST} \ i \end{bmatrix} \right\rangle \end{bmatrix} \\ \text{ARG-ST} & \langle \text{DP}_i [\text{COUNT} \ +] \rangle \end{bmatrix} \right\rangle \qquad \left\langle \text{book} , \begin{bmatrix} \textit{cntn-lxm} \\ \text{SYN} & \begin{bmatrix} \text{HEAD} & \begin{bmatrix} \textit{noun} \\ \text{AGR} \ \boxed{1} & \begin{bmatrix} \textit{plural} \\ \text{PER} \ \textit{3rd} \\ \text{NUM} \ \textit{pl} \end{bmatrix} \\ \text{CASE} \ \textit{acc} \end{bmatrix} \\ \text{VAL} & \begin{bmatrix} \text{SPR} & \langle [\text{AGR} \ \boxed{1}] \rangle \\ \text{COMPS} & \langle \rangle \\ \text{MOD} & \langle \rangle \end{bmatrix} \end{bmatrix} \\ \text{SEM} & \begin{bmatrix} \text{MODE} \ \textit{ref} \\ \text{INDEX} \ i \\ \text{RESTR} \ \left\langle \begin{bmatrix} \text{RELN} \ \textit{book} \\ \text{INST} \ i \end{bmatrix} \right\rangle \end{bmatrix} \\ \text{ARG-ST} & \langle \text{DP}_i [\text{COUNT} \ +] \rangle \end{bmatrix} \right\rangle$$

Q Technically, neither of these is a lexical sequence. Why?

A Lexical sequences must be **fully resolved**, and these don't fully resolve the DP on the ARG-ST and SPR lists.

'Family of Lexical Sequences'

- A pairing of orthographic form with some underspecified feature structure of type *lexeme* or *word*, which can be fully resolved to a number of different lexical sequences
- The book often refers to lexical entries, for example, as being 'families of lexical sequences', because they license infinitely many fully resolved lexical sequences
- So the examples on the previous slide are technically all families of lexical sequences, because the ARG-ST and VAL constituents are not fully specified

Lexical Terminology

- Lexeme
- Lexical entry
- Lexical sequence
- 'Family of lexical sequences'
- **Lexical rule**
- **Lexical rule instantiation**
- **Word structure**

Lexical Rule

- A part of the grammar that we write that allows us to derive...
  - inflected forms of words, given lexical entries
  - or additional lexical sequences where the feature structure is some subtype of *lexeme*
    - ...which can unify with the INPUT feature of additional lexical rules until we arrive at a lexical sequence pairing a word-form with a feature structure of type *word*

Lexical Rule: An Example

3rd-Singular Verb Lexical Rule

$$\left[\begin{array}{l} \textit{i-rule} \\ \textsc{input} \quad \left\langle \boxed{3}\ ,\ \left[\begin{array}{l}\textit{verb-lxm}\\ \textsc{sem}\ \left[\textsc{restr}\ \boxed{A}\right]\end{array}\right]\right\rangle \\ \textsc{output} \quad \left\langle \mathrm{F}_{3SG}\left(\boxed{3}\right),\ \left[\begin{array}{ll} \textsc{syn} & \left[\textsc{head}\ \left[\begin{array}{ll}\textsc{form} & \textit{fin}\\ \textsc{agr} & \textit{3sing}\end{array}\right]\right] \\ \textsc{sem} & \left[\textsc{restr}\ \boxed{A} \oplus \ldots\right] \\ \textsc{arg-st} & \left\langle\left[\textsc{case}\ \textit{nom}\right],\ \ldots\right\rangle \end{array}\right]\right\rangle \end{array}\right]$$

In plain English: take a verb lexeme, add the 3rd singular ending, and make it a finite word with a nominative subject that agrees with it.

Lexical Rule Instantiation

- A fully resolved feature structure that corresponds to one of those rules in the grammar

- That is, the rule with a particular lexical sequence as its INPUT and the corresponding lexical sequence as its OUTPUT
  - e.g. the 3rd-Singular Verb Lexical Rule with ⟨sleep, ...⟩ as INPUT and ⟨sleeps, ...⟩ as OUTPUT
  - one rule in the grammar; one instantiation per word it licenses

Word Structure

- The bottom part of the tree
- A fully resolved feature structure of type *word* over some string which is the spelling or pronunciation of the word
- e.g. *park* in *the cat sleeps in the park*

$$\begin{bmatrix} \textit{word} & & \\ \text{SYN} & \text{HEAD} & \begin{bmatrix} \textit{noun} & & \\ \text{AGR} & \boxed{1} & \begin{bmatrix} \textit{3sing} & \\ \text{PER} & \textit{3rd} \\ \text{NUM} & \textit{sg} \\ \text{GEND} & \textit{neut} \end{bmatrix} \\ \text{CASE} & \textit{acc} & \end{bmatrix} \\ & \text{VAL} & \begin{bmatrix} \text{SPR} & \langle \boxed{2} \rangle \\ \text{COMPS} & \langle\rangle \\ \text{MOD} & \langle\rangle \end{bmatrix} \\ \text{SEM} & & \begin{bmatrix} \text{MODE} & \textit{ref} \\ \text{INDEX} & j \\ \text{RESTR} & \left\langle \begin{bmatrix} \text{RELN} & \textit{park} \\ \text{INST} & j \end{bmatrix} \right\rangle \end{bmatrix} \\ \text{ARG-ST} & \langle \boxed{2} \rangle & \end{bmatrix}$$

park

Default Inheritance

Why do we have default inheritance?

Default Inheritance

Why do we have default inheritance?

A Generalizations with exceptions are common:

- Most nouns in English aren't marked for CASE, but pronouns are.
- Most verbs in English only distinguish two agreement categories (*3sing* and *non-3sing*), but ***be*** distinguishes more.
- Most prepositions in English are transitive, but ***here*** and ***there*** are intransitive.
- Most nominal words in English are 3rd person, but some (all of them pronouns) are 1st or 2nd person.
- Most proper nouns in English are singular, but some (mountain range names, sports team names) are plural.

Note: defaults are not necessary, and currently implemented grammars do not use default inheritance, though some work is looking into adding it.

Default Inheritance, Technicalities

If a type says $\left[\textsc{arg-st} \; / \left\langle \textsc{np} \right\rangle\right]$,

and one of its subtypes says $\left[\textsc{arg-st} \; \left\langle\right\rangle\right]$,

Default Inheritance, Technicalities

If a type says $\left[\text{ARG-ST } /\left\langle \text{NP} \right\rangle\right]$, and one of its subtypes says $\left[\text{ARG-ST } \left\langle\right\rangle\right]$, then the ARG-ST value of instances of the subtype is $\left\langle\right\rangle$.

In plain English: the ' / ' marks a default. A subtype can override it, and the subtype's value wins.

Default Inheritance, Technicalities

If a type says $\left[\textsc{arg-st} \; / \left\langle \text{NP} \right\rangle\right]$, and one of its subtypes says $\left[\textsc{arg-st} \; \left\langle\right\rangle\right]$, then the ARG-ST value of instances of the subtype is $\left\langle\right\rangle$.

If a type says $\left[\textsc{arg-st} \; \left\langle \text{NP} \right\rangle\right]$, and one of its subtypes says $\left[\textsc{arg-st} \; \left\langle\right\rangle\right]$,

Default Inheritance, Technicalities

If a type says $\left[\text{ARG-ST } /\left\langle \text{NP} \right\rangle\right]$,	and one of its subtypes says $\left[\text{ARG-ST } \left\langle\right\rangle\right]$,	then the ARG-ST value of instances of the subtype is $\left\langle\right\rangle$.
If a type says $\left[\text{ARG-ST } \left\langle \text{NP} \right\rangle\right]$,	and one of its subtypes says $\left[\text{ARG-ST } \left\langle\right\rangle\right]$,	then this subtype can have no instances, since they would have to satisfy contradictory constraints.

In plain English: without the ' / ' the constraint is inviolable. Nothing can be both $\left\langle \text{NP} \right\rangle$ and $\left\langle\right\rangle$, so the subtype is empty.

Default Inheritance, More Technicalities

- If a type says $\left[\text{MOD } / \left\langle \text{S} \right\rangle\right]$, and one of its subtypes says $\left[\text{MOD } \left\langle \left[\text{SPR } \left\langle \text{NP} \right\rangle\right] \right\rangle\right]$, then the MOD value of instances of the subtype is what?

Default Inheritance, More Technicalities

- If a type says $\left[\text{MOD } / \left\langle \text{S} \right\rangle\right]$, and one of its subtypes says $\left[\text{MOD } \left\langle \left[\text{SPR } \left\langle \text{NP} \right\rangle\right] \right\rangle\right]$, then the MOD value of instances of the subtype is what?

$$\left[\text{MOD } \left\langle \begin{bmatrix} \text{HEAD} & / \textit{verb} \\ \text{SPR} & \left\langle \text{NP} \right\rangle \\ \text{COMPS} & / \left\langle\right\rangle \end{bmatrix} \right\rangle\right]$$

- That is, default constraints are 'pushed down'

- S abbreviates $\begin{bmatrix} \text{HEAD} & \textit{verb} \\ \text{SPR} & \left\langle\right\rangle \\ \text{COMPS} & \left\langle\right\rangle \end{bmatrix}$, so the subtype only overrides the part it mentions (SPR). The rest of the default survives, still as a default.

Question on Default Inheritance

Q Can a grammar rule override a default constraint on a word?

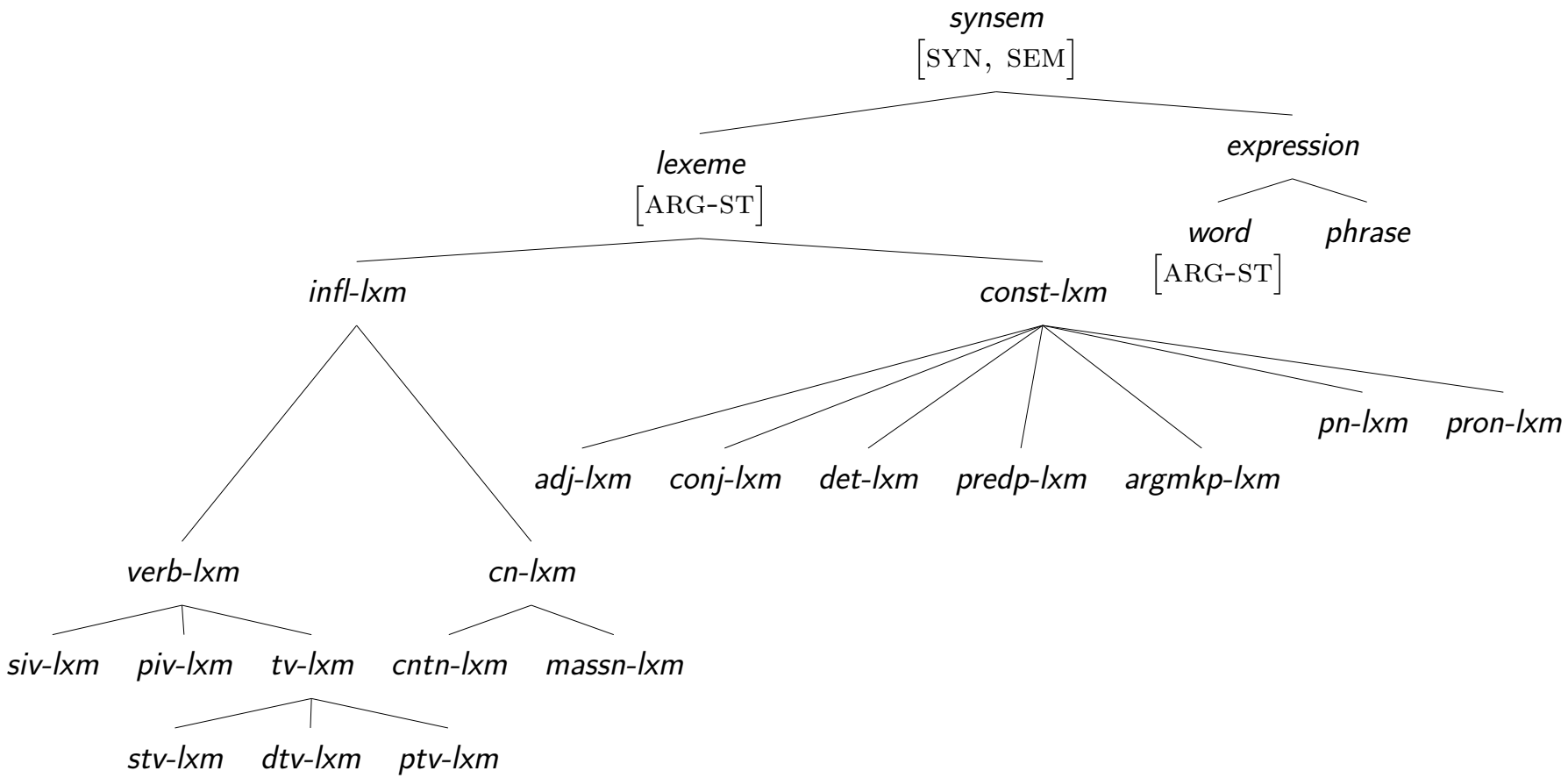
Question on Default Inheritance

Q Can a grammar rule override a default constraint on a word?

A No. Defaults are all 'cached out' in the lexicon.

- Words as used to build sentences have only inviolable constraints.
  - i.e. nodes of type *word* in your tree should never be shown with a defeasible constraint (no ' / ' in a tree)

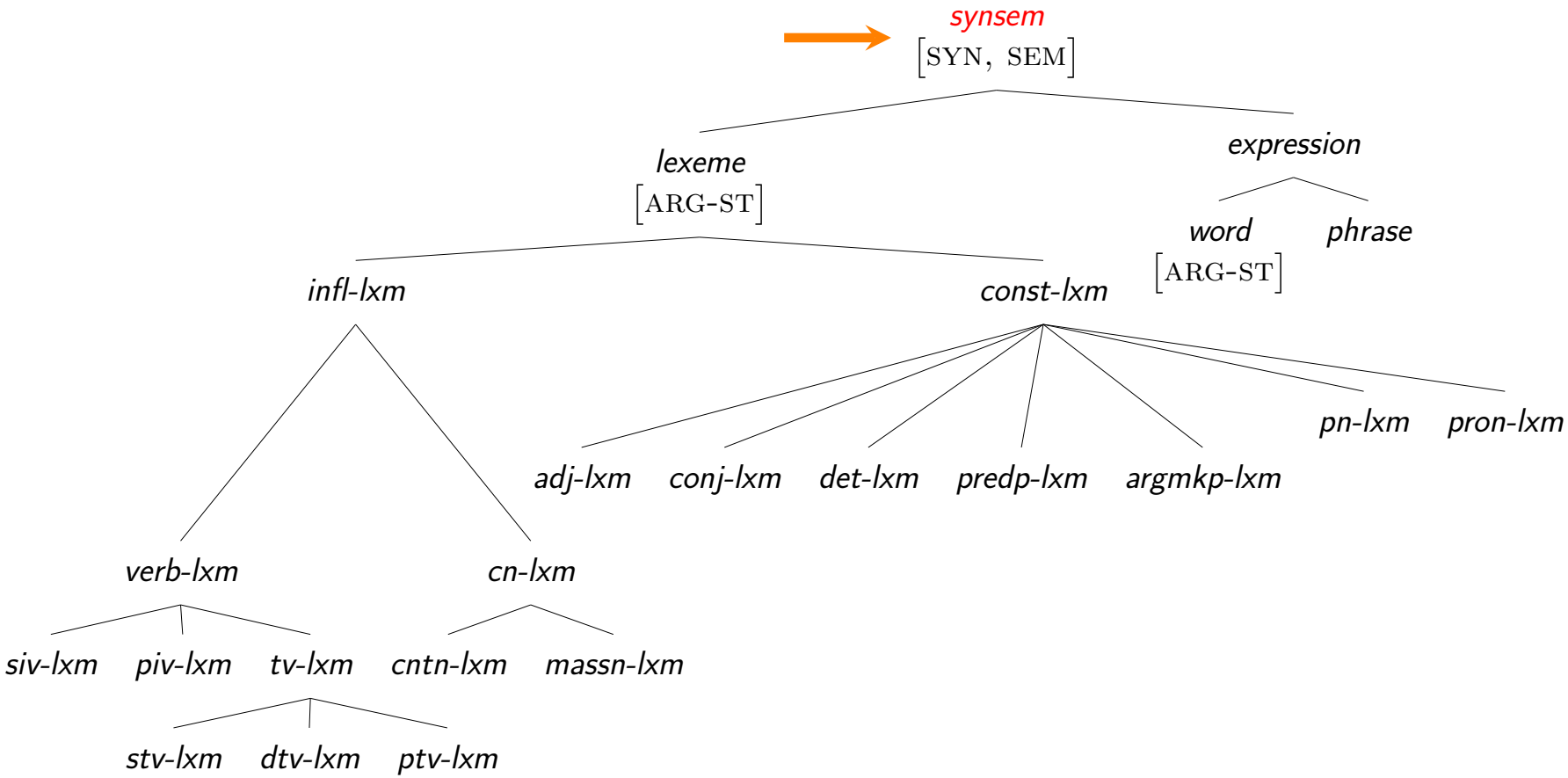
Our Lexeme Hierarchy



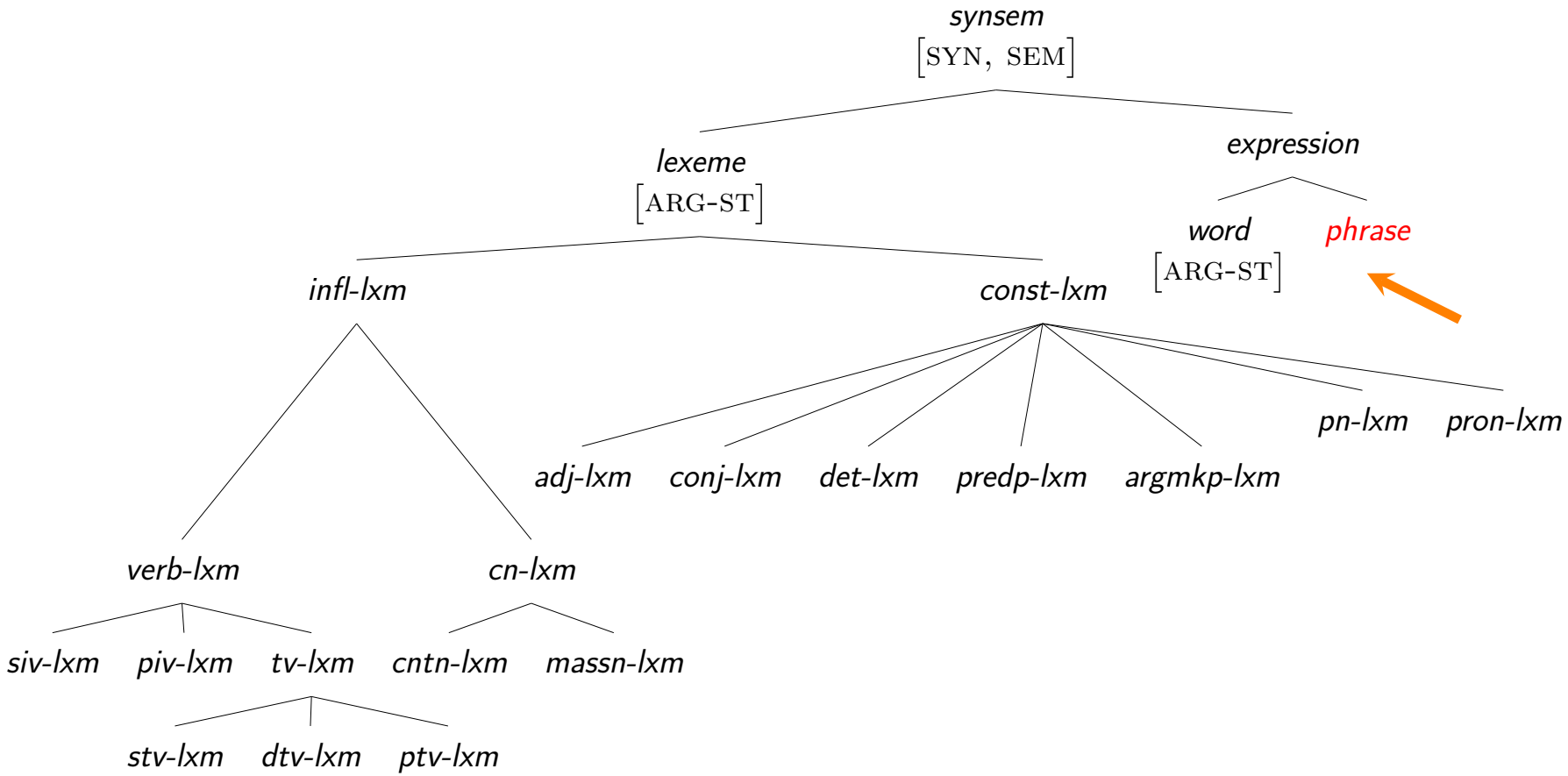
Functions of Types

- Stating what features are appropriate for what categories
- Stating generalizations
  - Constraints that apply to (almost) all instances
  - Generalizations about selection — where instances of that type can appear

Every *synsem* has the Features SYN and SEM

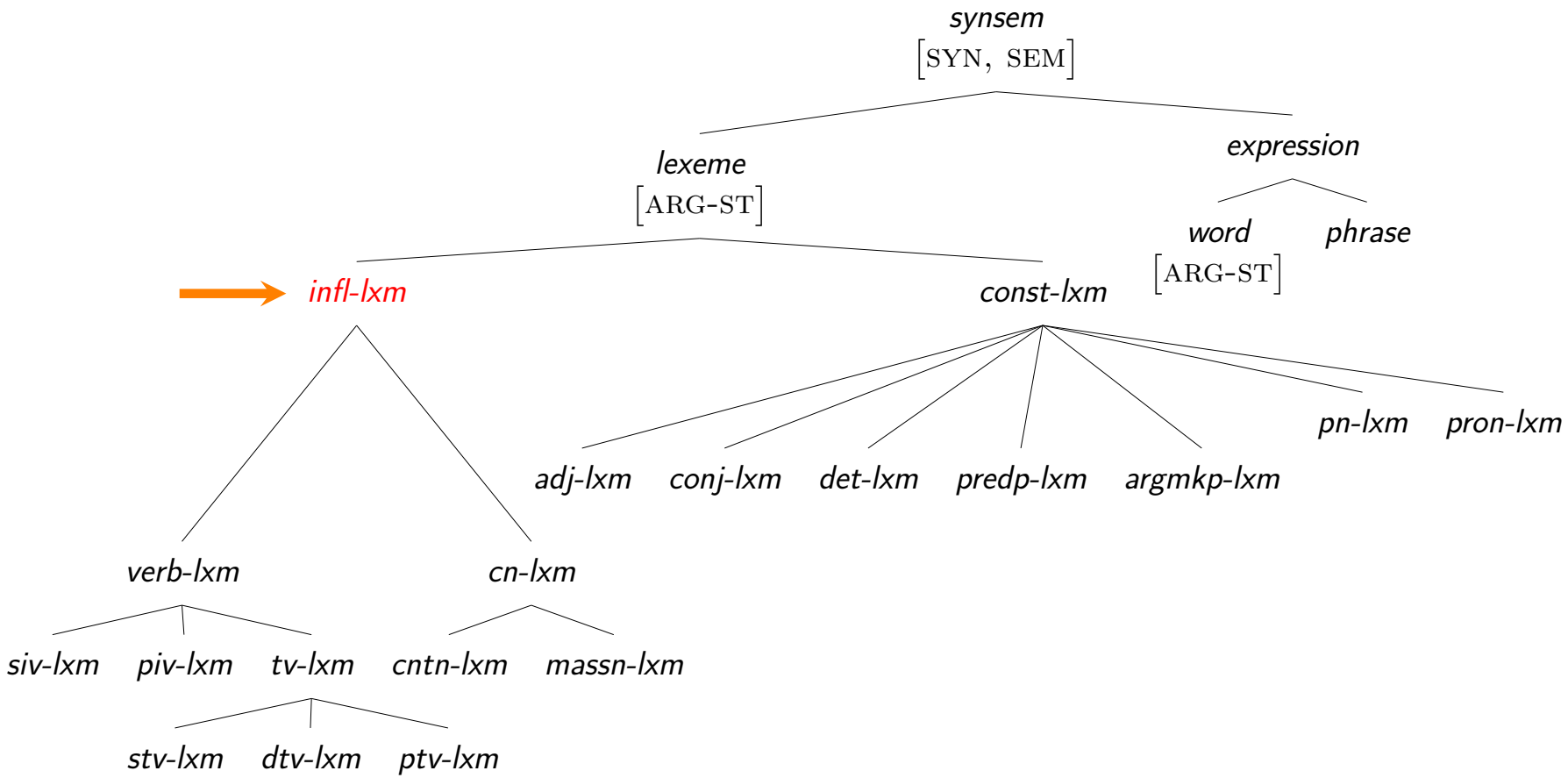


No ARG-ST on *phrase*



ARG-ST is declared on *lexeme* and on *word*, so only lexical items have it; a phrase never does.

A Constraint on *infl-lxm*: the SHAC

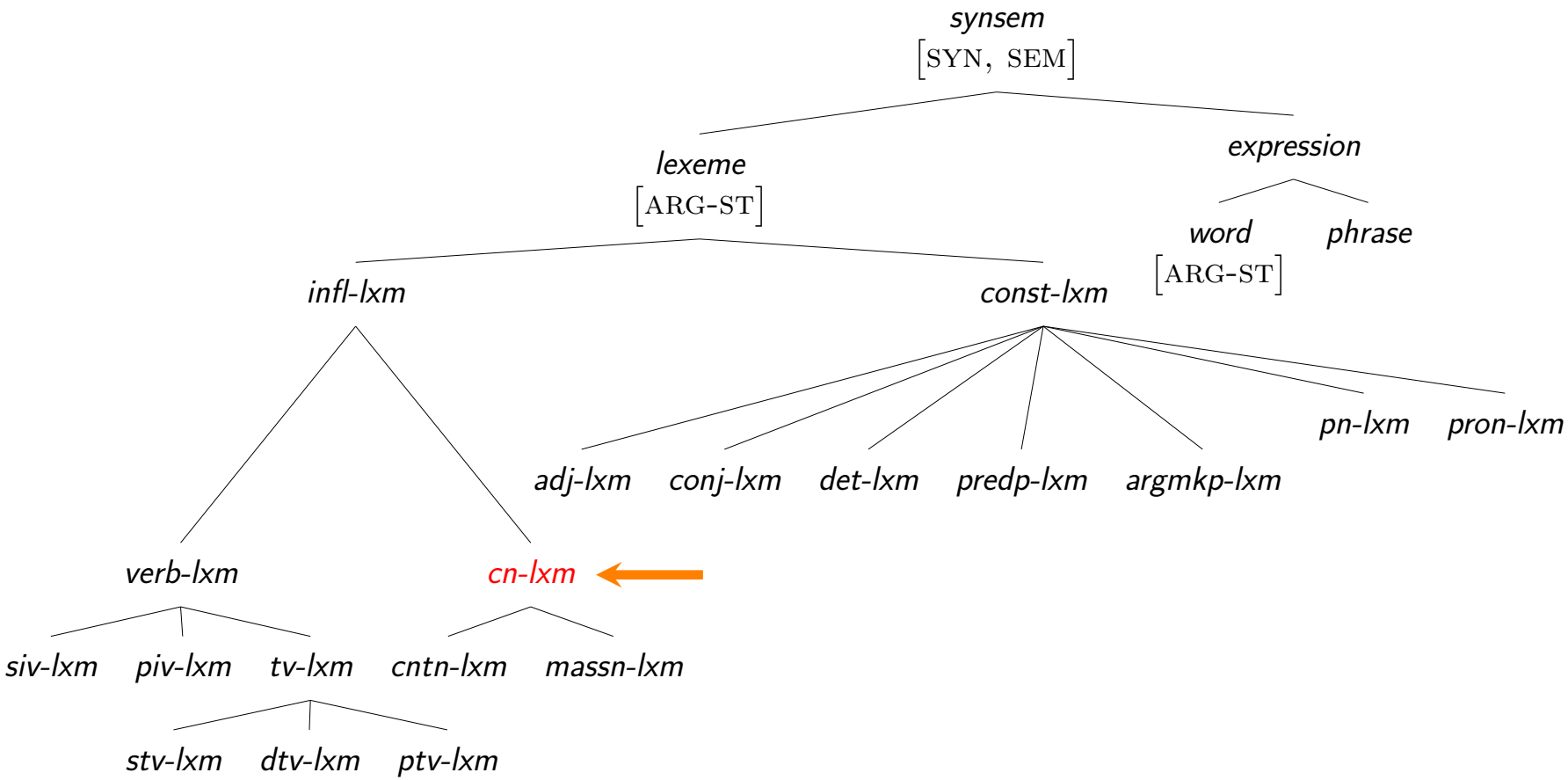


A Constraint on *infl-lxm*: the SHAC

$$\textit{infl-lxm}: \begin{bmatrix} \text{SYN} & \begin{bmatrix} \text{VAL} & \left[\text{SPR} \left\langle \left[\text{AGR } \boxed{1}\right] \right\rangle\right] \\ \text{HEAD} & \left[\text{AGR } \boxed{1}\right] \end{bmatrix} \end{bmatrix}$$

In plain English: every inflecting lexeme (nouns and verbs) has the same AGR value as its specifier. Stated once here, it is inherited by every noun and verb in the lexicon.

Constraints on *cn-lxm*

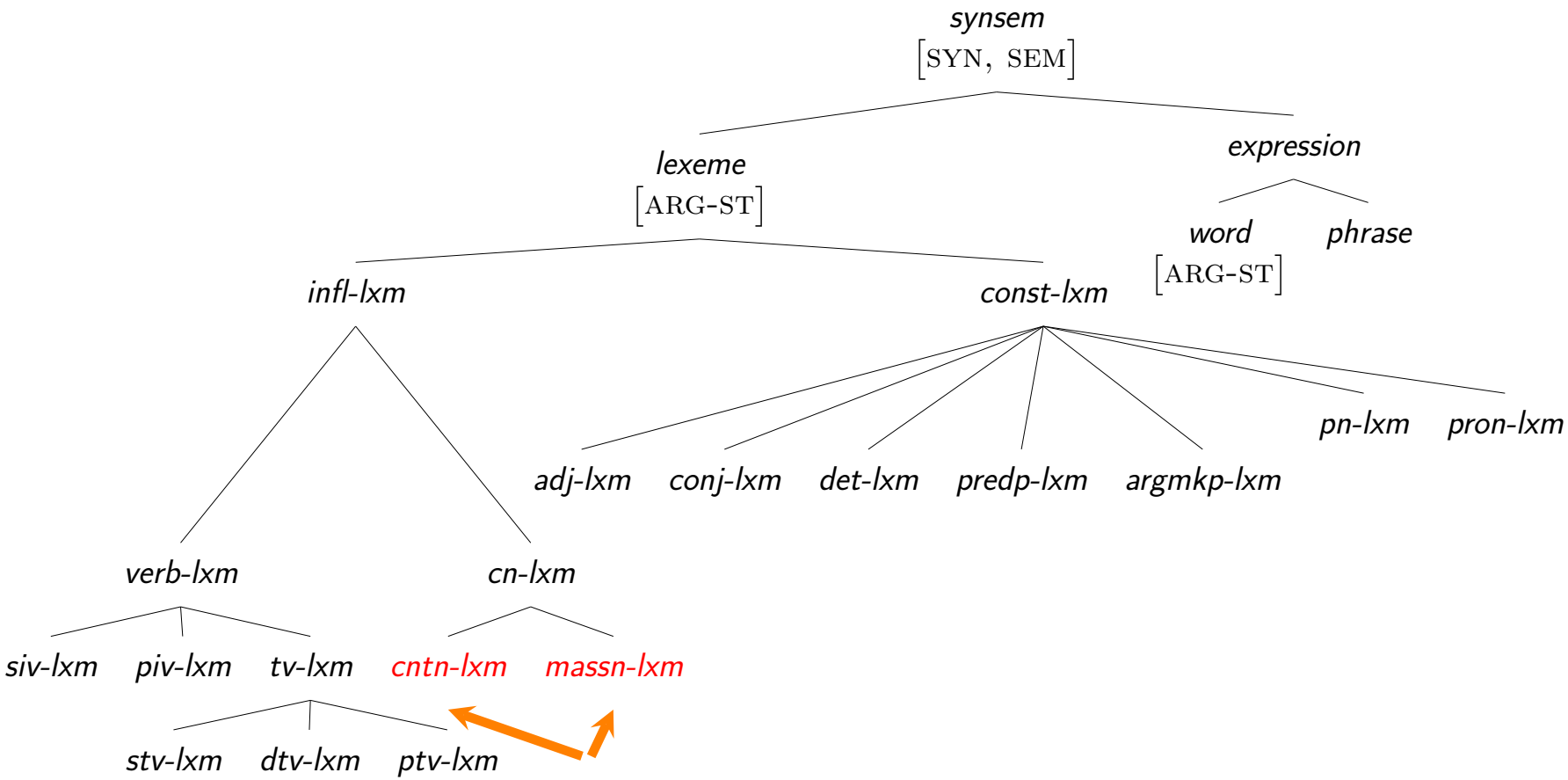


Constraints on *cn-lxm*

cn-lxm:
$$\begin{bmatrix} \text{SYN} & \begin{bmatrix} \text{HEAD} & \begin{bmatrix} \textit{noun} \\ \text{AGR} & \begin{bmatrix} \text{PER} & \textit{3rd} \end{bmatrix} \end{bmatrix} \\ \text{VAL} & \begin{bmatrix} \text{SPR} & \left\langle \begin{bmatrix} \text{HEAD} & \textit{det} \\ \text{INDEX} & \textit{i} \end{bmatrix} \right\rangle \end{bmatrix} \end{bmatrix} \\ \text{SEM} & \begin{bmatrix} \text{MODE} & / \ \textit{ref} \\ \text{INDEX} & \textit{i} \end{bmatrix} \\ \text{ARG-ST} & \langle \text{X} \rangle \oplus \ / \ \langle\rangle \end{bmatrix}$$

In plain English: a common noun is 3rd person, takes a determiner as its specifier, shares its index with that determiner, is normally referential, and normally has no arguments beyond the specifier.

Formally Distinguishing Count vs. Mass Nouns



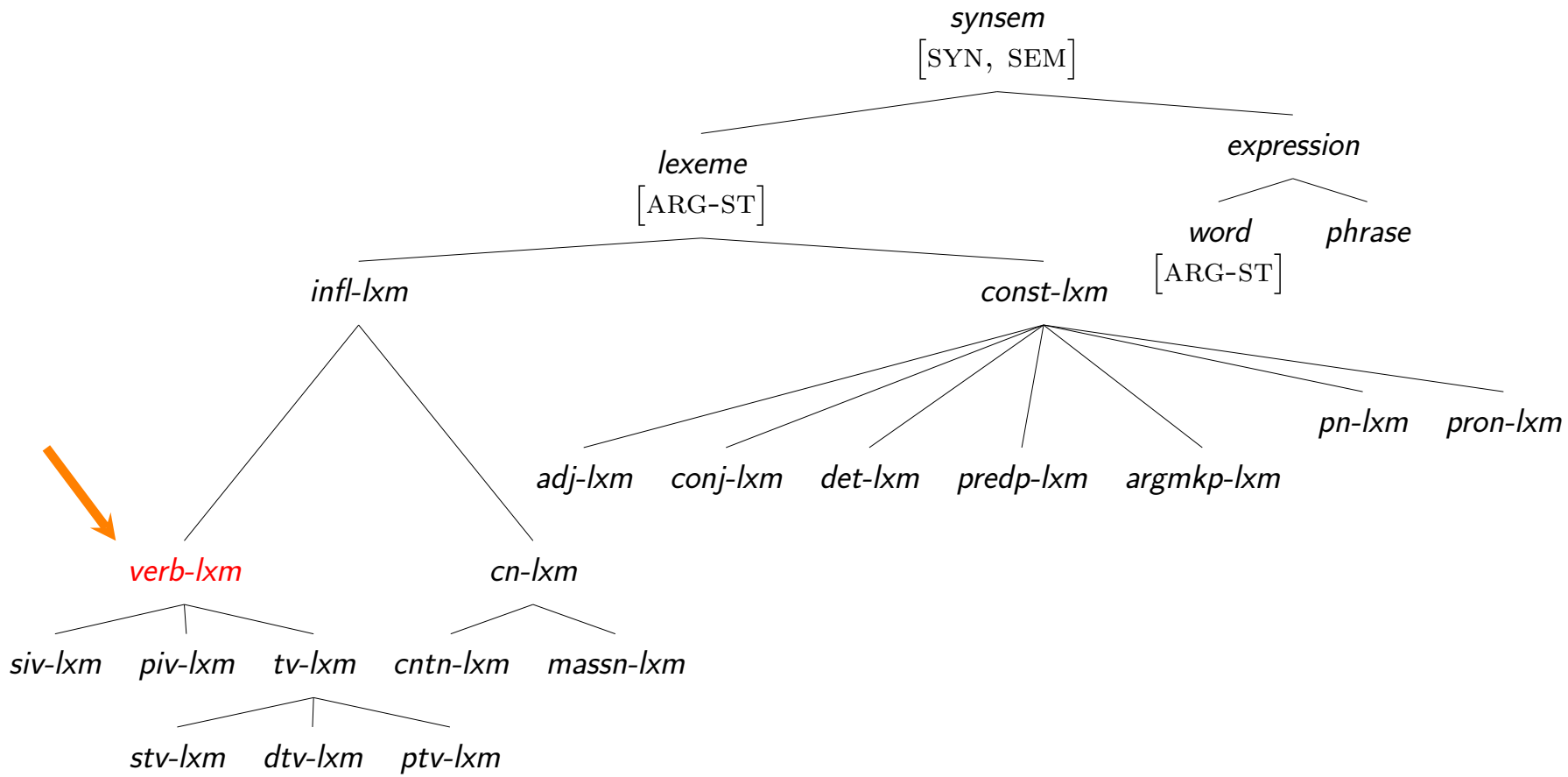
Formally Distinguishing Count vs. Mass Nouns

cntn-lxm: $\left[\text{SYN}\ \left[\text{VAL}\ \left[\text{SPR}\ \left\langle\left[\text{COUNT}\ +\right]\right\rangle\right]\right]\right]$

massn-lxm: $\left[\text{SYN}\ \left[\text{VAL}\ \left[\text{SPR}\ \left\langle\left[\text{COUNT}\ -\right]\right\rangle\right]\right]\right]$

The only difference is which determiners they take (*a book*, *much furniture*)

Constraints on *verb-lxm*

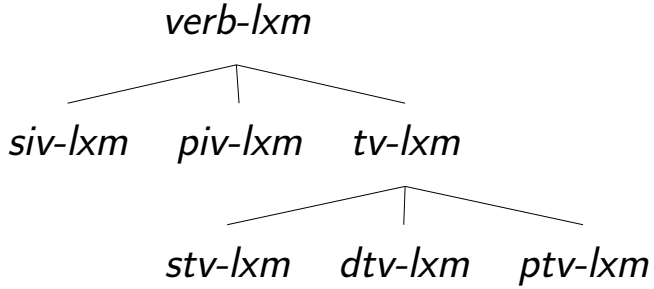


Constraints on *verb-lxm*

$$\textit{verb-lxm}: \begin{bmatrix} \textsc{syn} & \left[\textsc{head}\ \textit{verb}\right] \\ \textsc{sem} & \left[\textsc{mode}\ \textit{prop}\right] \\ \textsc{arg-st} & / \left\langle \textsc{np},\ \ldots \right\rangle \end{bmatrix}$$

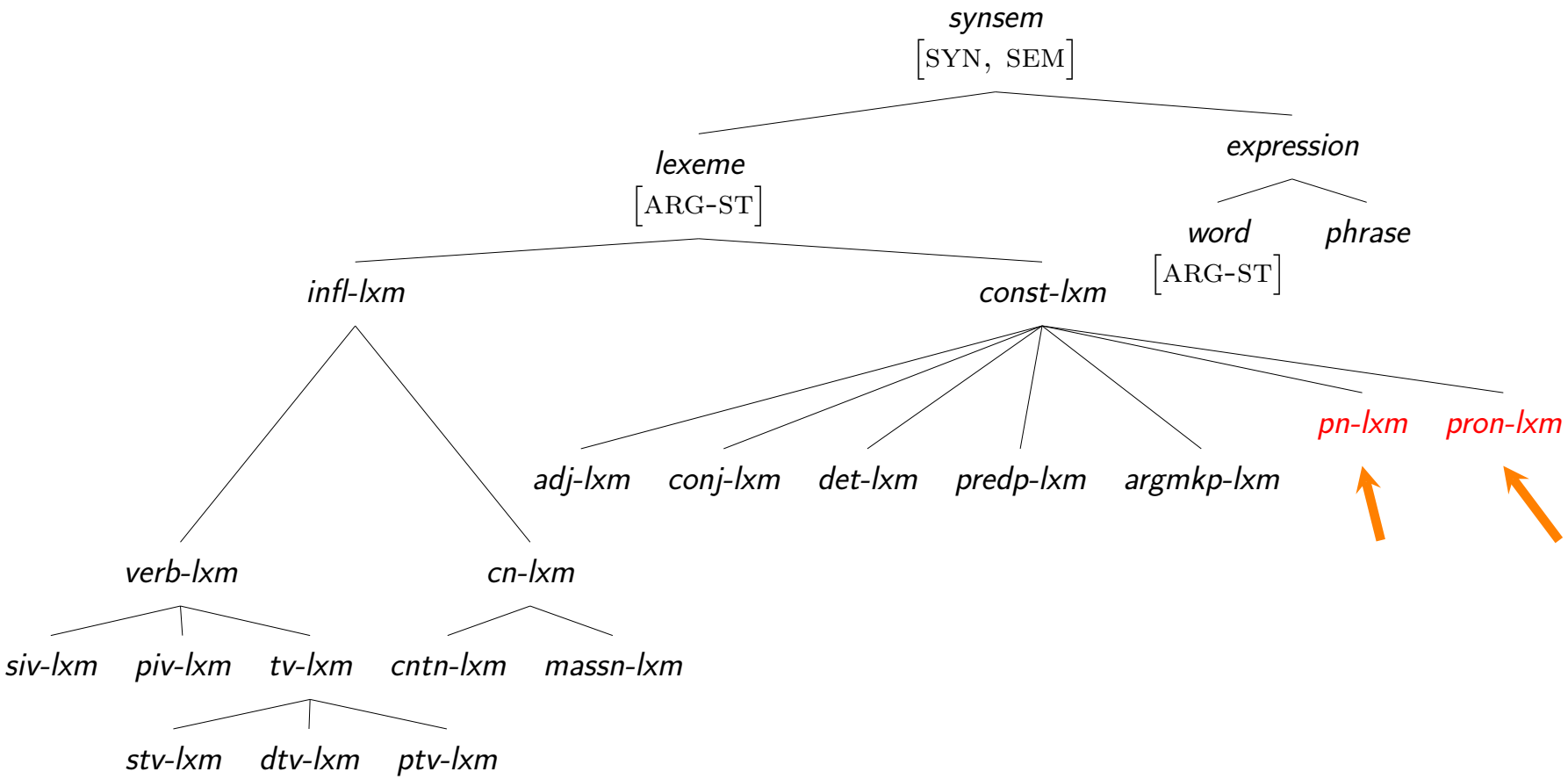
In plain English: a verb lexeme has HEAD *verb*, expresses a proposition, and by default its first argument (its subject) is an NP. The default lets a few verbs take other subjects (*That Kim left surprised me*).

Subtypes of *verb-lxm*



- *verb-lxm*: $\left[\textsc{arg-st}\ /\left\langle \mathrm{NP},\ \ldots\right\rangle\right]$
  - *siv-lxm*: $\left[\textsc{arg-st}\ \left\langle \mathrm{NP}\right\rangle\right]$ (strict intransitive: *sleep*)
  - *piv-lxm*: $\left[\textsc{arg-st}\ \left\langle \mathrm{NP},\ \mathrm{PP}\right\rangle\right]$ (prepositional intransitive: *rely*)
  - *tv-lxm*: $\left[\textsc{arg-st}\ \left\langle \mathrm{NP},\ \mathrm{NP},\ \ldots\right\rangle\right]$ (transitive)
    - *stv-lxm*: $\left[\textsc{arg-st}\ \left\langle \mathrm{NP},\ \mathrm{NP}\right\rangle\right]$ (strict transitive: *devour*)
    - *dtv-lxm*: $\left[\textsc{arg-st}\ \left\langle \mathrm{NP},\ \mathrm{NP},\ \mathrm{NP}\right\rangle\right]$ (ditransitive: *hand*)
    - *ptv-lxm*: $\left[\textsc{arg-st}\ \left\langle \mathrm{NP},\ \mathrm{NP},\ \mathrm{PP}\right\rangle\right]$ (prepositional transitive: *put*)

Proper Nouns and Pronouns



Proper Nouns and Pronouns

$$\textit{pn-lxm}: \begin{bmatrix} \textsc{syn} & \begin{bmatrix} \textsc{head} & \begin{bmatrix} \textit{noun} \\ \textsc{agr} & \begin{bmatrix} \textsc{per} & \textit{3rd} \\ \textsc{num} & \textit{/ sg} \end{bmatrix} \end{bmatrix} \end{bmatrix} \\ \textsc{sem} & \begin{bmatrix} \textsc{mode} & \textit{ref} \end{bmatrix} \\ \textsc{arg-st} & \textit{/} \langle\rangle \end{bmatrix}$$

$$\textit{pron-lxm}: \begin{bmatrix} \textsc{syn} & \begin{bmatrix} \textsc{head} & \textit{noun} \end{bmatrix} \\ \textsc{sem} & \begin{bmatrix} \textsc{mode} & \textit{/ ref} \end{bmatrix} \\ \textsc{arg-st} & \langle\rangle \end{bmatrix}$$

In plain English: proper nouns are 3rd person, usually singular (but *the Alps*) and usually take no specifier (but *the Kim I know*). Pronouns are nouns with no arguments, usually referential (but *itself* is $\begin{bmatrix} \textsc{mode} & \textit{ana} \end{bmatrix}$).

The Case Constraint

An outranked NP is $\left[\textsc{case}\ \textit{acc}\right]$.

Which of these NPs does it make accusative?

- object of verb
- second object of verb
- object of argument-marking preposition
- object of predicational preposition

The Case Constraint

An outranked NP is $\left[\text{CASE } \textit{acc}\right]$.

Which of these NPs does it make accusative?

- object of verb ✓
- second object of verb ✓
- object of argument-marking preposition ✓
- object of predicational preposition (✓)

Everything after the first element of an ARG-ST list is outranked. The object of an argument-marking preposition has the same rank as its PP, so it is outranked by the verb's subject too; the object of a predicational preposition is outranked within the preposition's own ARG-ST.

The Case Constraint, Continued

An outranked NP is $\left[\text{CASE } \textit{acc}\right]$.

- Subjects of verbs
  - Should we add a clause to cover nominative subjects, e.g. un-outranked NPs are $\left[\text{CASE } \textit{nom}\right]$?
- Any other instances of case marking in English?
- Does it apply to case systems in other languages?

The Case Constraint, Continued

An outranked NP is $\left[\text{CASE } \textit{acc}\right]$.

- Subjects of verbs
  - Should we add a clause to cover nominative subjects, e.g. un-outranked NPs are $\left[\text{CASE } \textit{nom}\right]$?
    * No. *We expect them to leave.* (Chapter 12)
      them is the SPR of ***leave*** and a COMPS member of ***expect***, so this would conflict
    * Lexical rules for finite verbs will handle nominative subjects.

- Any other instances of case marking in English?
  Possessives: *my*, *Kim's* (Chapter 6, Problem 4).

- Does it apply to case systems in other languages?
  No: the Case Constraint is an English-specific constraint.

Apparent Redundancy of *pos* and *lexeme*

- Why do we need both the *pos* subhierarchy and lexeme types?

- *pos*:
  - Applies to words and phrases; models the relationship between them
    e.g. 'verbs head VPs', 'nouns head NPs'
  - Constrains which features are appropriate (no AUX on *noun*)

- *lexeme*:
  - Generalizations about combinations of constraints

Lexical Types & Lexical Rules

- Lexical entries capture the similarities among ***run, runs, running,*** and ***ran***.

- The lexical type hierarchy captures the similarities among ***run, sleep,*** and ***laugh***, among those and other verbs like ***devour*** and ***hand***, and among those and other words like ***book***.

  – What do ***devour*** and ***book*** have in common? The SHAC

- Lexical rules capture the similarities among ***runs, sleeps, devours, hands,***...

Activity: All the Constraints on *give*

Here is the lexical entry for ***give***. Which types does it inherit from, and what does each one add?

$$\left\langle \text{give} \ , \begin{bmatrix} \textit{dtv-lxm} & \\ \text{SEM} & \begin{bmatrix} \text{INDEX} & s \\ \text{RESTR} & \left\langle \begin{bmatrix} \text{RELN} & \textit{give} \\ \text{SIT} & s \\ \text{GIVER} & i \\ \text{RECIPIENT} & j \\ \text{GIFT} & k \end{bmatrix} \right\rangle \end{bmatrix} \\ \text{ARG-ST} & \langle \text{X}_i, \ \text{Y}_j, \ \text{Z}_k \rangle \end{bmatrix} \right\rangle$$

Activity: All the Constraints on *give*

Here is the lexical entry for ***give***. Which types does it inherit from, and what does each one add?

$$\left\langle \text{give} \ , \begin{bmatrix} \textit{dtv-lxm} & & \\ \text{SEM} & \begin{bmatrix} \text{INDEX} & s & \\ \text{RESTR} & \left\langle \begin{bmatrix} \text{RELN} & \textit{give} \\ \text{SIT} & s \\ \text{GIVER} & i \\ \text{RECIPIENT} & j \\ \text{GIFT} & k \end{bmatrix} \right\rangle \end{bmatrix} \\ \text{ARG-ST} & \langle \text{X}_i, \ \text{Y}_j, \ \text{Z}_k \rangle & \end{bmatrix} \right\rangle$$

- *dtv-lxm* $\subset$ *tv-lxm*: $\left[\text{ARG-ST} \ \langle \text{NP}, \ \text{NP}, \ \text{NP} \rangle\right]$
- *verb-lxm*: $\left[\text{HEAD} \ \textit{verb}\right]$, $\left[\text{MODE} \ \textit{prop}\right]$, $\left[\text{ARG-ST} \ / \langle \text{NP}, \ \ldots \rangle\right]$
- *infl-lxm*: the SHAC
- *lexeme*, *synsem*: the features ARG-ST, SYN and SEM are appropriate

Step 1: *dtv-lxm* and *tv-lxm*

$$\left\langle \text{give} \ , \begin{bmatrix} \textit{dtv-lxm} & & \\ \text{SEM} & \begin{bmatrix} \text{INDEX} & s & \\ \text{RESTR} & \left\langle \begin{bmatrix} \text{RELN} & \textit{give} \\ \text{SIT} & s \\ \text{GIVER} & i \\ \text{RECIPIENT} & j \\ \text{GIFT} & k \end{bmatrix} \right\rangle \end{bmatrix} \\ \text{ARG-ST} & \langle \text{NP}_i, \ \text{NP}_j, \ \text{NP}_k \rangle & \end{bmatrix} \right\rangle$$

dtv-lxm says the three arguments are all NPs: the three variables X, Y, Z are resolved to NP.

Step 2: *verb-lxm*

$$\left\langle \text{give} \ , \begin{bmatrix} \textit{dtv-lxm} & \\ \text{SYN} & \left[\text{HEAD} \ \textit{verb}\right] \\ \text{SEM} & \begin{bmatrix} \text{MODE} & \textit{prop} \\ \text{INDEX} & s \\ \text{RESTR} & \left\langle \begin{bmatrix} \text{RELN} & \textit{give} \\ \text{SIT} & s \\ \text{GIVER} & i \\ \text{RECIPIENT} & j \\ \text{GIFT} & k \end{bmatrix} \right\rangle \end{bmatrix} \\ \text{ARG-ST} & \langle \text{NP}_i, \ \text{NP}_j, \ \text{NP}_k \rangle \end{bmatrix} \right\rangle$$

verb-lxm adds $\left[\text{HEAD} \ \textit{verb}\right]$ and $\left[\text{MODE} \ \textit{prop}\right]$. Its default $\left[\text{ARG-ST} \ / \left\langle \text{NP}, \ \ldots \right\rangle\right]$ is already satisfied, so it adds nothing new.

Step 3: *infl-lxm* (the SHAC)

$$\left\langle \text{give} \ , \begin{bmatrix} \textit{dtv-lxm} & & \\ \text{SYN} & \begin{bmatrix} \text{HEAD} & \begin{bmatrix} \textit{verb} \\ \text{AGR} \ \boxed{1} \end{bmatrix} \\ \text{VAL} & \left[\text{SPR} \ \left\langle \left[\text{AGR} \ \boxed{1}\right] \right\rangle\right] \end{bmatrix} \\ \text{SEM} & \begin{bmatrix} \text{MODE} & \textit{prop} \\ \text{INDEX} & s \\ \text{RESTR} & \left\langle \begin{bmatrix} \text{RELN} & \textit{give} \\ \text{SIT} & s \\ \text{GIVER} & i \\ \text{RECIPIENT} & j \\ \text{GIFT} & k \end{bmatrix} \right\rangle \end{bmatrix} \\ \text{ARG-ST} & \langle \text{NP}_i, \ \text{NP}_j, \ \text{NP}_k \rangle \end{bmatrix} \right\rangle$$

The SHAC identifies the verb's AGR with its specifier's. Nothing yet links SPR to ARG-ST: the ARP is a constraint on *word*, so it applies only after a lexical rule.

Activity: All the Constraints on *give*

$$\left\langle \text{give} \ , \left[\begin{array}{ll} \textit{dtv-lxm} & \\ \text{SYN} & \left[\begin{array}{ll} \text{HEAD} & \left[\begin{array}{l} \textit{verb} \\ \text{AGR} \ \boxed{1} \end{array}\right] \\ \text{VAL} & \left[\text{SPR} \ \left\langle \left[\text{AGR} \ \boxed{1}\right] \right\rangle\right] \end{array}\right] \\ \text{SEM} & \left[\begin{array}{ll} \text{MODE} & \textit{prop} \\ \text{INDEX} & s \\ \text{RESTR} & \left\langle \left[\begin{array}{ll} \text{RELN} & \textit{give} \\ \text{SIT} & s \\ \text{GIVER} & i \\ \text{RECIPIENT} & j \\ \text{GIFT} & k \end{array}\right] \right\rangle \end{array}\right] \\ \text{ARG-ST} & \left\langle \text{NP}_i, \ \text{NP}_j, \ \text{NP}_k \right\rangle \end{array}\right] \right\rangle$$

Compare the entry we started from: HEAD, VAL, MODE and the NP types all came from the hierarchy. The entry itself only had to say the form, the meaning and the type.

Overview

- Motivation for lexical hierarchy
- Lexical terminology
- Default inheritance
- Tour of the lexeme hierarchy
- The Case Constraint
- *pos* vs. *lexeme*
- In-class activity: all the constraints on ***give***

Lexical Rules

Overview

- How lexical rules fit in
- Three types of lexical rules, constraints
- Example: Plural noun lexical rule
- Advice on writing lexical rules
- Constant lexemes and the other lexical rules (so far)
- ARG-ST & ARP; the feature FORM
- Activity: the word structure for ***writers***

Lexical Entries, Lexical Types & Lexical Rules

- Lexical entries capture the similarities among ***run, runs, running,*** and ***ran***.
- The lexical type hierarchy captures the similarities among ***run, sleep,*** and ***laugh***, among those and other verbs like ***devour*** and ***hand***, and among those and other words like ***book***.
  - What do ***devour*** and ***book*** have in common? The SHAC
- Lexical rules capture the similarities among ***runs, sleeps, devours, hands,***...

How Lexical Rules Fit In: Well-Formed Tree Structure

Φ is a well-formed tree structure according to grammar G if and only if:

1. Φ is a tree structure,

2. the label of Φ's root node satisfies the constraint $\begin{bmatrix} \text{SYN} & \begin{bmatrix} \text{HEAD} & \begin{bmatrix} \textit{verb} \\ \text{FORM} \ \ \textit{fin} \end{bmatrix} \\ \text{VAL} & \begin{bmatrix} \text{COMPS} \ \ \langle\rangle \\ \text{SPR} \ \ \langle\rangle \end{bmatrix} \end{bmatrix} \end{bmatrix}$

   and

3. each local subtree within Φ is either phrasally licensed or lexically licensed.

A sentence is a finite clause with nothing missing, where every branching is licensed by a rule and every leaf by the lexicon.

How Lexical Rules Fit In: Lexical Sequences

$\langle \omega, \phi \rangle$ is a lexical sequence if and only if ω is a phonological form (an atom), ϕ is a feature structure, and either:

1. G contains some lexical entry $\langle d_1, d_2 \rangle$ such that ω satisfies d_1 and ϕ satisfies d_2, or

2. there is some lexical rule instantiation licensed by G (a feature structure of type *l-rule*) whose OUTPUT value is $\langle \omega, \phi \rangle$.

A form paired with a feature structure is a lexical sequence if it comes straight from a lexical entry, or is the output of a lexical rule. The second clause is recursive: the rule's input must itself be a lexical sequence, so rules can feed each other.

How Lexical Rules Fit In: Lexical Licensing

A word structure of the form on the right is licensed if and only if:

1. $\langle \omega, \phi \rangle$ is a lexical sequence, where ϕ is of type *word*,
2. (Case Constraint:) an outranked NP is [CASE acc], and
3. ϕ satisfies the Binding Theory.

$$\begin{array}{c} \phi \\ | \\ \omega \end{array}$$

Only lexical sequences of type *word* can appear in trees. Lexemes have to pass through at least one lexical rule to become words: that is what the inflectional rules are for.

Parsimony & Plausibility

- Lexical rules capture **productive** generalizations.
  - When a new word enters the lexicon, it is assigned a lexical type, and we expect every lexical rule that applies to that type to apply to the new word too.
- There may be some 'precompiling' going on as well.
  - Psycholinguistically, very frequent words may just be stored ready-made rather than derived each time.

Three Kinds of Lexical Rules (very English-specific!)

- **Inflectional**: *lexeme* to *word* (takes us 'over the boundary' from lexeme to word)

  Examples?

- **Derivational**: *lexeme* to *lexeme*

  Examples?

- **Post-Inflectional**: *word* to *word*
  (Chapters 11, 13, 14)

Three Kinds of Lexical Rules (very English-specific!)

- **Inflectional**: *lexeme* to *word* (takes us 'over the boundary' from lexeme to word)

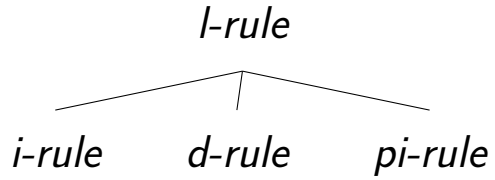
  Examples?
  Singular Noun LR, Plural Noun LR, 3rd-Singular Verb LR, Non-3rd-Singular Verb LR, Past-Tense Verb LR, Base Form LR, Constant Lexeme LR

- **Derivational**: *lexeme* to *lexeme*

  Examples?
  Present Participle LR, Past Participle LR, Agent Nominalization LR

- **Post-Inflectional**: *word* to *word*
  (Chapters 11, 13, 14)

Three Subtypes of *l-rule*



$$\textit{l-rule}: \begin{bmatrix} \textsc{input} & \textit{l-sequence} \left\langle \textsc{x} \ , \begin{bmatrix} \textsc{sem} & / \boxed{2} \end{bmatrix} \right\rangle \\ \textsc{output} & \textit{l-sequence} \left\langle \textsc{y} \ , \begin{bmatrix} \textsc{sem} & / \boxed{2} \end{bmatrix} \right\rangle \end{bmatrix}$$

$$\textit{i-rule}: \begin{bmatrix} \textsc{input} & \left\langle \textsc{x} \ , \begin{bmatrix} \textit{lexeme} & \\ \textsc{syn} & \boxed{3} \\ \textsc{arg-st} & \boxed{A} \end{bmatrix} \right\rangle \\ \textsc{output} & \left\langle \textsc{y} \ , \begin{bmatrix} \textit{word} & \\ \textsc{syn} & \boxed{3} \\ \textsc{arg-st} & \boxed{A} \end{bmatrix} \right\rangle \end{bmatrix} \qquad \textit{d-rule}: \begin{bmatrix} \textsc{input} & \left\langle \textsc{x} \ , \begin{bmatrix} \textit{lexeme} & \\ \textsc{syn} & / \boxed{3} \end{bmatrix} \right\rangle \\ \textsc{output} & \left\langle \textsc{y} \ , \begin{bmatrix} \textit{lexeme} & \\ \textsc{syn} & / \boxed{3} \end{bmatrix} \right\rangle \end{bmatrix}$$

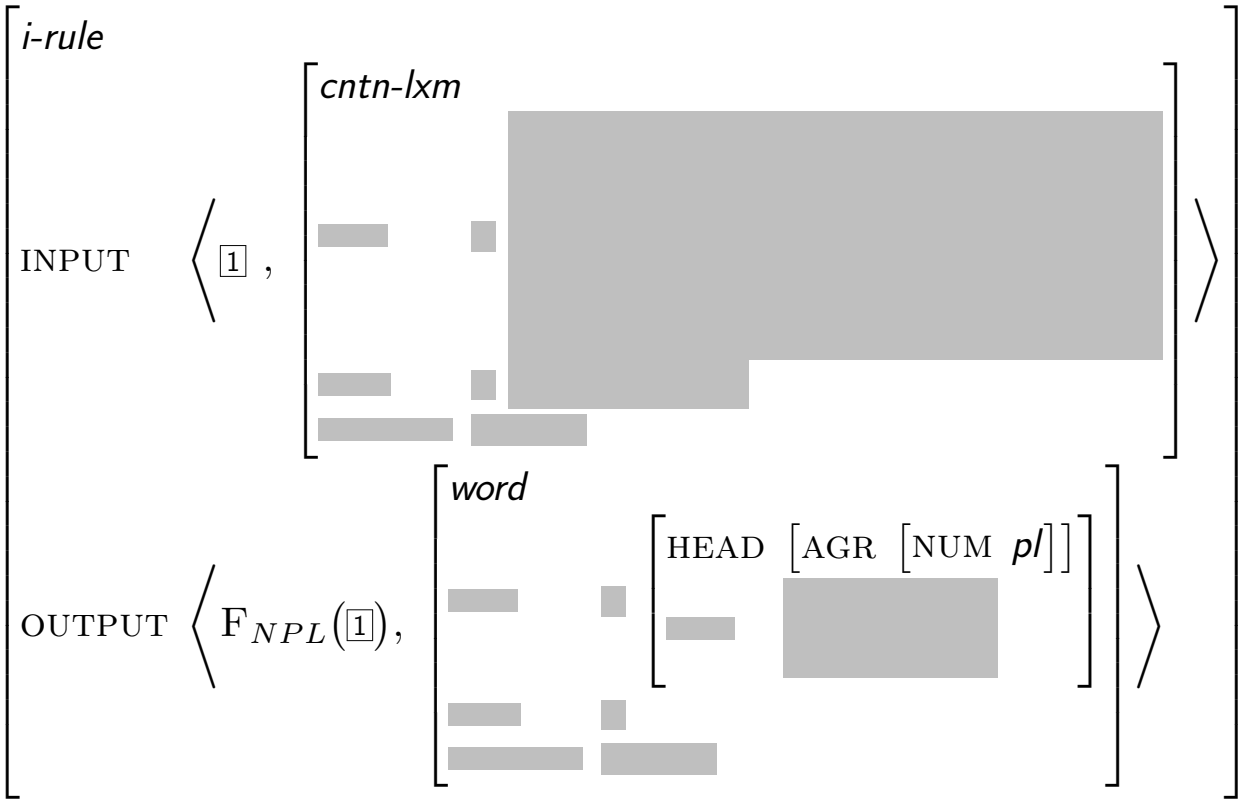
l-rule: the output keeps the input's meaning, by default. *i-rule*: the output is a *word* with the same SYN and ARG-ST, no exceptions. *d-rule*: the output is another *lexeme* with the same SYN, by default.

Plural Noun LR

$$\left[\begin{array}{l} \textit{i-rule} \\ \textsc{input} \quad \langle \boxed{1} \ , \ \textit{cntn-lxm} \rangle \\ \textsc{output} \quad \left\langle \mathrm{F}_{NPL}(\boxed{1}), \ \left[\begin{array}{l} \textit{word} \\ \textsc{syn} \ \left[\textsc{head} \ \left[\textsc{agr} \ \left[\textsc{num} \ \textit{pl}\right]\right]\right] \end{array}\right] \right\rangle \end{array}\right]$$

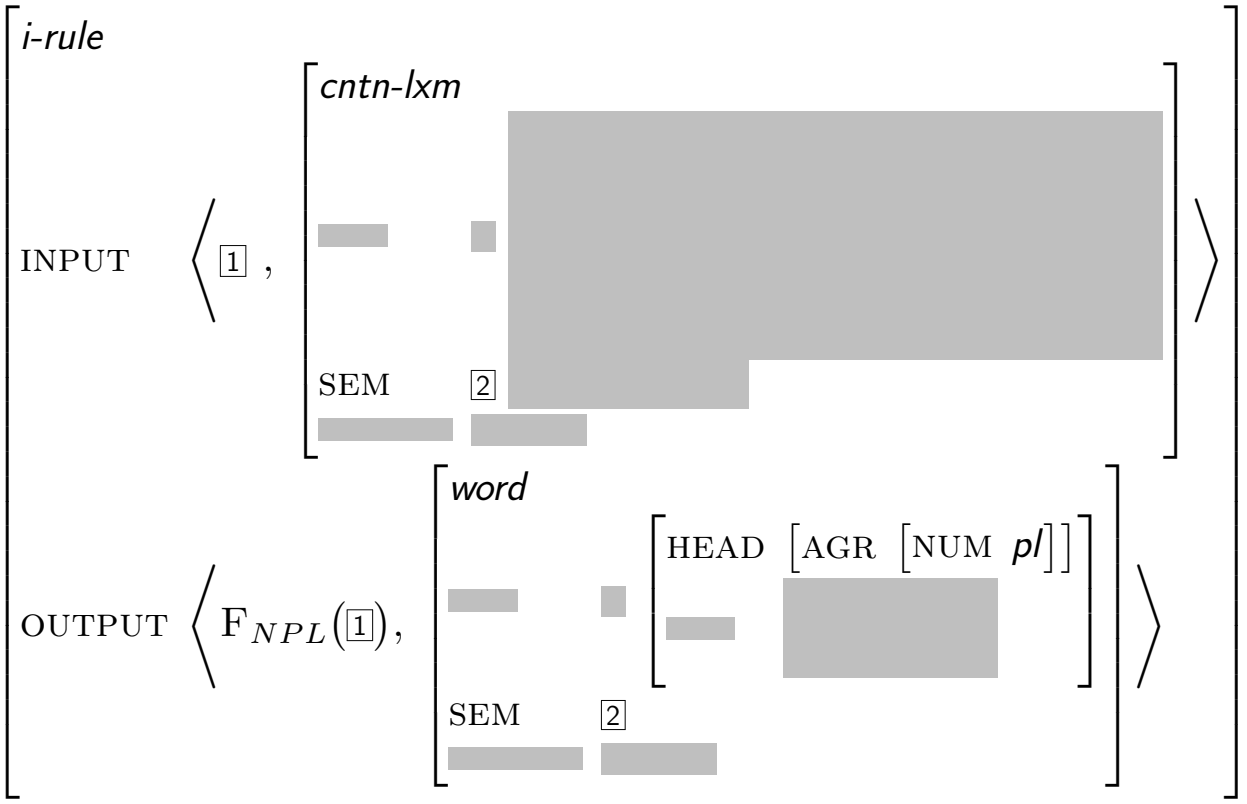
Take any count noun lexeme $\boxed{1}$, add the plural ending to its form, and make it a word whose number is plural. That is all the rule itself says; everything else comes from the types.

Plural Noun LR with Inherited Constraints



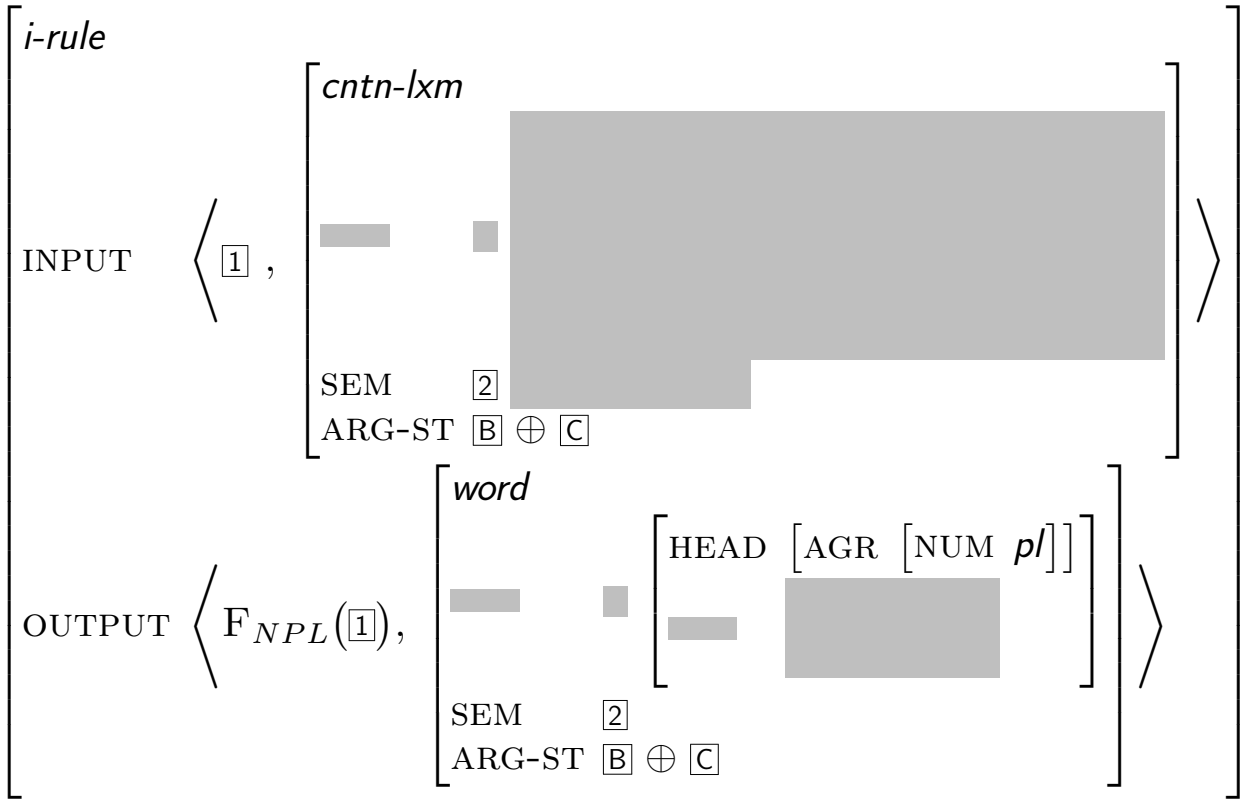
What else do we know? The types *i-rule*, *cntn-lxm* and *word* all add constraints.

Plural Noun LR with Inherited Constraints — *I-rule*



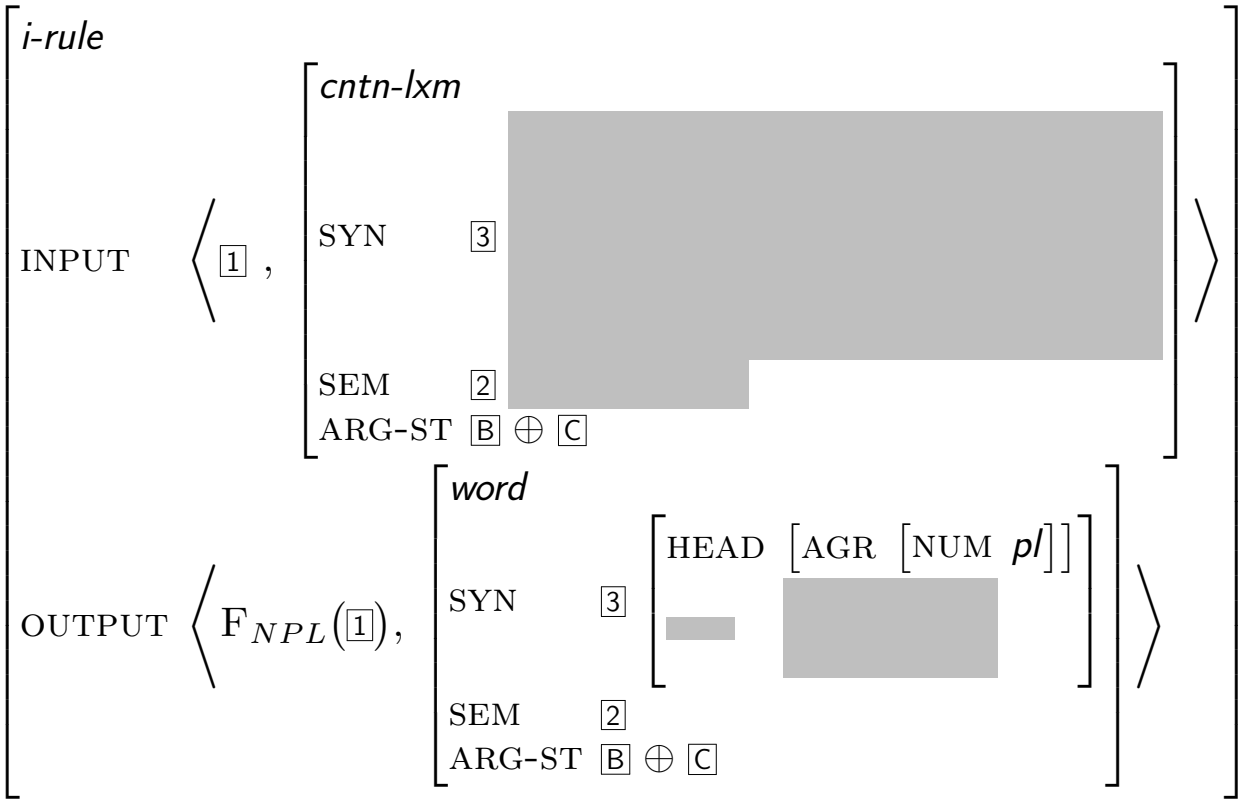
I-rule: the output's SEM is the input's, by default. The plural rule says nothing about meaning, so the default holds.

Plural Noun LR with Inherited Constraints — *i-rule*



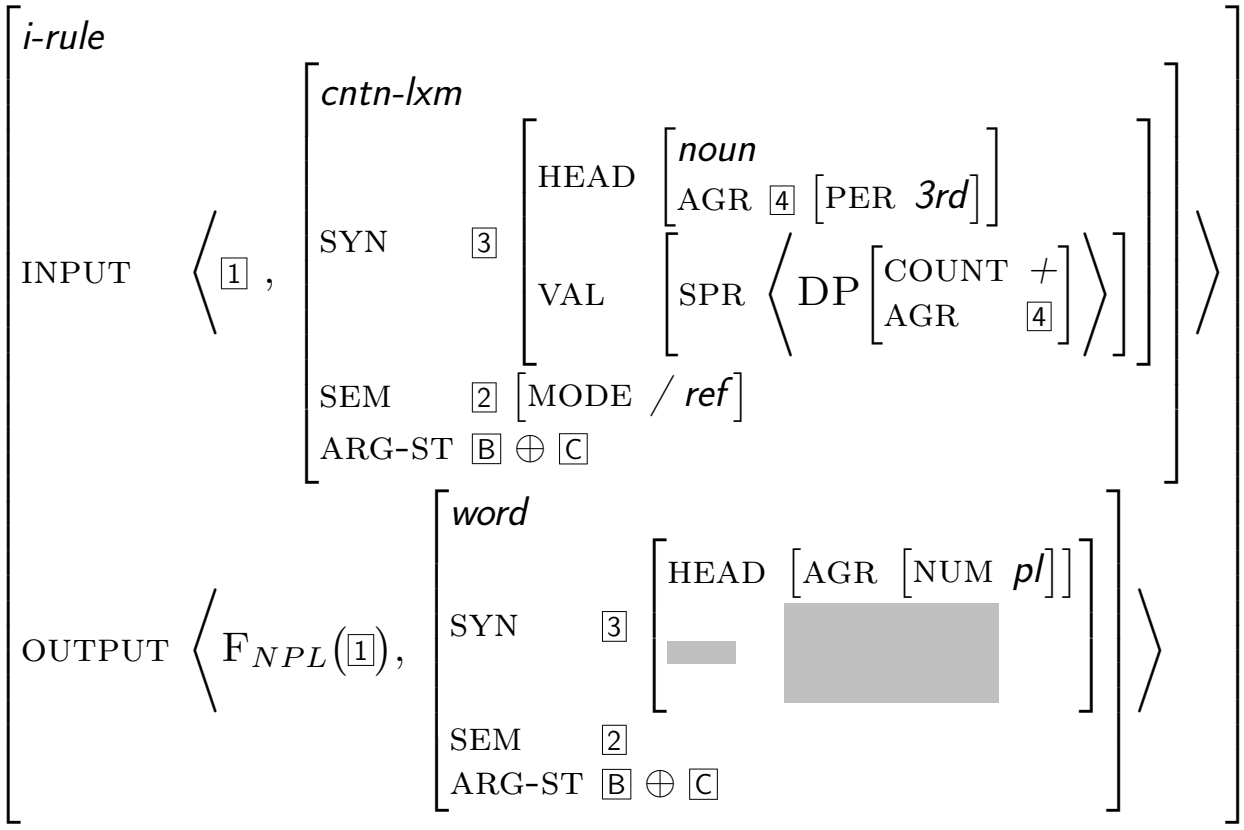
i-rule: the output is of type *word* and has the same ARG-ST as the input.

Plural Noun LR with Inherited Constraints — *i-rule*



i-rule: and the same SYN, so the [NUM pl] the rule adds is unified with the input's SYN.

Plural Noun LR — *cntn-lxm*, *cn-lxm*, *infl-lxm*



cntn-lxm: [COUNT +]. *cn-lxm*: *noun*, [PER 3rd], a DP specifier, [MODE / ref]. *infl-lxm*: the SHAC, 4.

Plural Noun LR with Inherited Constraints — *word*

$$\left[\begin{array}{l}\textit{i-rule}\\ \text{INPUT}\quad \left\langle \boxed{1}\,,\ \left[\begin{array}{ll}\textit{cntn-lxm}&\\ \text{SYN}&\boxed{3}\left[\begin{array}{ll}\text{HEAD}&\left[\begin{array}{l}\textit{noun}\\ \text{AGR}\ \boxed{4}\ \left[\text{PER}\ \textit{3rd}\right]\end{array}\right]\\ \text{VAL}&\left[\text{SPR}\ \left\langle \text{DP}\left[\begin{array}{ll}\text{COUNT}&+\\ \text{AGR}&\boxed{4}\end{array}\right]\right\rangle\right]\end{array}\right]\\ \text{SEM}&\boxed{2}\left[\text{MODE}\ /\ \textit{ref}\right]\\ \text{ARG-ST}&\boxed{B}\oplus\boxed{C}\end{array}\right]\right\rangle\\ \text{OUTPUT}\quad \left\langle \text{F}_{NPL}\left(\boxed{1}\right),\ \left[\begin{array}{ll}\textit{word}&\\ \text{SYN}&\boxed{3}\left[\begin{array}{ll}\text{HEAD}&\left[\text{AGR}\ \left[\text{NUM}\ \textit{pl}\right]\right]\\ \text{VAL}&\left[\begin{array}{ll}\text{SPR}&\boxed{B}\\ \text{COMPS}&\boxed{C}\end{array}\right]\end{array}\right]\\ \text{SEM}&\boxed{2}\\ \text{ARG-ST}&\boxed{B}\oplus\boxed{C}\end{array}\right]\right\rangle\end{array}\right]$$

word: the ARP splits ARG-ST into SPR and COMPS. Only the output is a *word*, so only it gets this.

Practicalities — Applying Lexical Rules

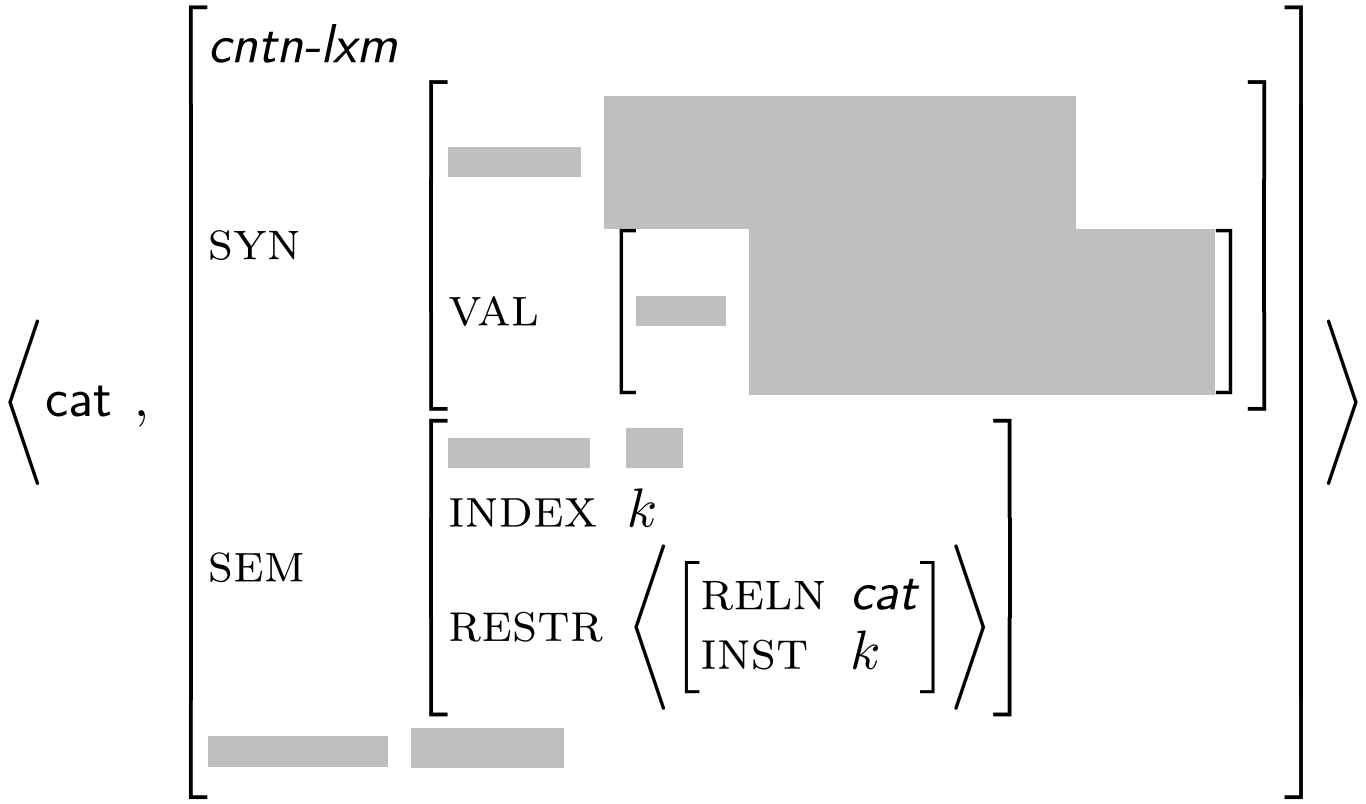
- INPUT is a family of lexical sequences.
- OUTPUT is another family of lexical sequences.
  - ...usually a smaller family
  - ...usually a disjoint one
- The only differences between the families are those stipulated in the rule (or the rule's type).
- Similarities are handled by the constraints on *l-rule* and its subtypes.
- If we've written the LRs correctly, nothing is left underconstrained.

Example: Lexical Entry for *cat*

$$\left\langle \text{cat} \ , \begin{bmatrix} \textit{cntn-lxm} \\ \text{SEM} \begin{bmatrix} \text{INDEX} & k \\ \text{RESTR} & \left\langle \begin{bmatrix} \text{RELN} & \textit{cat} \\ \text{INST} & k \end{bmatrix} \right\rangle \end{bmatrix} \end{bmatrix} \right\rangle$$

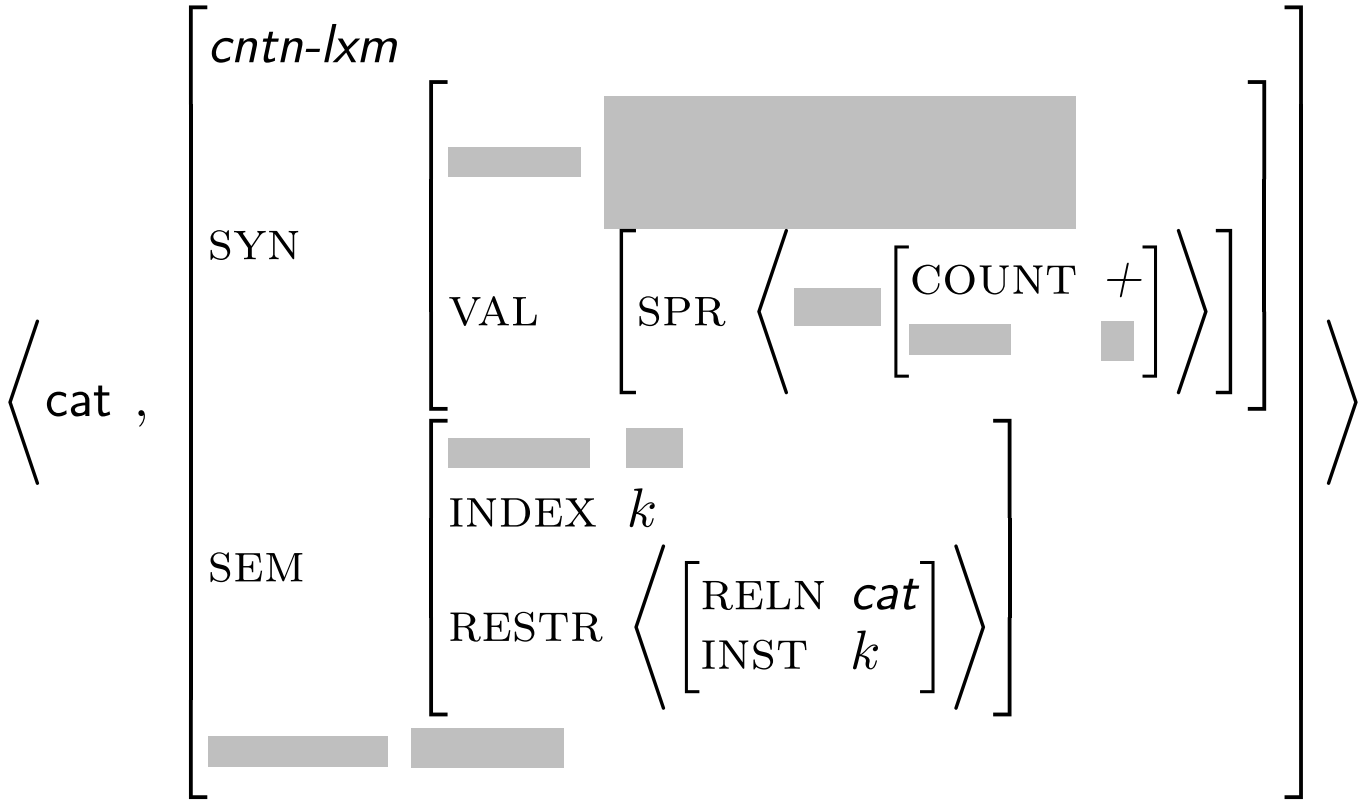
The entry gives only the form, the type and the meaning.

Example: *cat*, with Inheritance



The types fill in the rest. Which type adds what?

Example: *cat*, with Inheritance — *cntn-lxm*



cntn-lxm: the specifier is [COUNT +].

Example: *cat*, with Inheritance — *cn-lxm*

$$\left\langle \text{cat} \ , \begin{bmatrix} \textit{cntn-lxm} & & \\ \text{SYN} & \text{HEAD} & \begin{bmatrix} \textit{noun} \\ \text{AGR} \ \boxed{} \ \begin{bmatrix} \text{PER} & \textit{3rd} \end{bmatrix} \end{bmatrix} \\ & \text{VAL} & \begin{bmatrix} \text{SPR} \left\langle \text{DP} \begin{bmatrix} \text{COUNT} & + \\ \boxed{} & \boxed{} \end{bmatrix} \right\rangle \end{bmatrix} \\ \text{SEM} & & \begin{bmatrix} \text{MODE} & \textit{ref} \\ \text{INDEX} & k \\ \text{RESTR} & \left\langle \begin{bmatrix} \text{RELN} & \textit{cat} \\ \text{INST} & k \end{bmatrix} \right\rangle \end{bmatrix} \\ \text{ARG-ST} & \langle \text{X} \rangle & \end{bmatrix} \right\rangle$$

cn-lxm: *noun*, [PER 3rd], a DP specifier, [MODE ref] and one argument. **Defeasible constraints are inviolable in lexical entries!** The type says [MODE / ref]; nothing in the entry overrides it, so for ***cat*** it is simply [MODE ref].

Example: *cat*, with Inheritance — *infl-lxm*

$$\left\langle \text{cat} \ , \begin{bmatrix} \textit{cntn-lxm} & & \\ \text{SYN} & \text{HEAD} & \begin{bmatrix} \textit{noun} \\ \text{AGR} \ \boxed{7} \ \begin{bmatrix} \text{PER} \ \textit{3rd} \end{bmatrix} \end{bmatrix} \\ & \text{VAL} & \begin{bmatrix} \text{SPR} \ \left\langle \text{DP} \begin{bmatrix} \text{COUNT} & + \\ \text{AGR} & \boxed{7} \end{bmatrix} \right\rangle \end{bmatrix} \\ \text{SEM} & \begin{bmatrix} \text{MODE} & \textit{ref} \\ \text{INDEX} & k \\ \text{RESTR} & \left\langle \begin{bmatrix} \text{RELN} & \textit{cat} \\ \text{INST} & k \end{bmatrix} \right\rangle \end{bmatrix} \\ \text{ARG-ST} & \langle \text{X} \rangle & \end{bmatrix} \right\rangle$$

infl-lxm: the SHAC identifies the noun's AGR with its specifier's, $\boxed{7}$.

Plural Noun LR

$$\left[\begin{array}{l} \textit{i-rule} \\ \textsc{input} \quad \langle \boxed{1} \ , \ \textit{cntn-lxm} \rangle \\ \textsc{output} \quad \left\langle \mathrm{F}_{NPL}(\boxed{1}), \left[\begin{array}{l} \textit{word} \\ \textsc{syn} \ \left[\textsc{head} \ \left[\textsc{agr} \ \left[\textsc{num} \ \textit{pl}\right]\right]\right] \end{array}\right] \right\rangle \end{array}\right]$$

Now feed ***cat*** to the rule.

Licensing *cats*: the Plural Noun LR's Own Contributions

$$\left[\begin{array}{l}\textit{i-rule}\\ \text{INPUT}\quad \left\langle \boxed{1}\text{cat}\ ,\ \left[\begin{array}{l}\textit{cntn-lxm}\\ \text{SYN}\quad \boxed{3}\ \left[\begin{array}{ll}\text{HEAD} & \left[\begin{array}{l}\textit{noun}\\ \text{AGR}\ \boxed{7}\ \left[\text{PER}\ \textit{3rd}\right]\end{array}\right]\\ \text{VAL} & \left[\text{SPR}\ \left\langle \text{DP}\left[\begin{array}{ll}\text{COUNT} & +\\ \text{AGR} & \boxed{7}\end{array}\right]\right\rangle\right]\end{array}\right]\\ \text{SEM}\quad \boxed{2}\ \left[\begin{array}{l}\text{MODE}\ \ \textit{ref}\\ \text{INDEX}\ \ k\\ \text{RESTR}\ \left\langle\left[\begin{array}{ll}\text{RELN} & \textit{cat}\\ \text{INST} & k\end{array}\right]\right\rangle\end{array}\right]\\ \text{ARG-ST}\ \boxed{B}\ \langle \text{X}\rangle \oplus \boxed{C}\ \langle\rangle\end{array}\right]\right\rangle\\ \text{OUTPUT}\quad \left\langle \text{F}_{NPL}\left(\boxed{1}\right)\ ,\ \left[\begin{array}{l}\textit{word}\\ \text{SYN}\quad \boxed{3}\ \left[\begin{array}{ll}\text{HEAD} & \left[\text{AGR}\ \left[\text{NUM}\ \ \textit{pl}\right]\right]\\ \text{VAL} & \left[\begin{array}{ll}\text{SPR} & \boxed{B}\\ \text{COMPS} & \boxed{C}\end{array}\right]\end{array}\right]\\ \text{SEM}\quad \boxed{2}\\ \text{ARG-ST}\ \boxed{B}\oplus\boxed{C}\end{array}\right]\right\rangle\end{array}\right]$$

In blue: what the rule and its output type add; the rest is copied or inherited.

cats: The (Family of) Lexical Sequence(s)

$$\left\langle \text{cats} \;,\; \left[\begin{array}{ll} \textit{word} & \\ \text{SYN} & \left[\begin{array}{ll} \text{HEAD} & \left[\begin{array}{l} \textit{noun} \\ \text{AGR}\ \boxed{7}\ \textit{3pl} \end{array}\right] \\ \text{VAL} & \left[\begin{array}{ll} \text{SPR} & \boxed{\text{B}} \left\langle \text{DP} \left[\begin{array}{ll} \text{COUNT} & + \\ \text{AGR} & \boxed{7} \end{array}\right] \right\rangle \\ \text{COMPS} & \langle\rangle \end{array}\right] \end{array}\right] \\ \text{SEM} & \left[\begin{array}{ll} \text{MODE} & \textit{ref} \\ \text{INDEX} & k \\ \text{RESTR} & \left\langle \left[\begin{array}{ll} \text{RELN} & \textit{cat} \\ \text{INST} & k \end{array}\right] \right\rangle \end{array}\right] \\ \text{ARG-ST} & \boxed{\text{B}} \end{array}\right] \right\rangle$$

Strictly a family: the DP in SPR is not fully resolved, so this stands for every lexical sequence that fills it in.

Practicalities — Writing Lexical Rules

- Determine the type of the LR.
- Determine the class of possible inputs.
- Determine what should change.
  - If INPUT and OUTPUT values are identified (by default or otherwise) and only OUTPUT value is mentioned, then...
    information is added. e.g. Plural Noun LR

    (Lexical sequences incompatible with that value are not possible inputs)
  - If INPUT and OUTPUT values are identified by default, but different values are given on the INPUT and OUTPUT of the rule, then...
    information is changed. e.g. Agent Nominalization LR
  - If INPUT and OUTPUT values are identified by an inviolable constraint, but different values are given on the INPUT and OUTPUT of the rule, then...
    there is no well-formed input: the rule can never apply.

Constant Lexemes

- What kinds of words are constant lexemes in our grammar?
- Why do we need a rule for these words?
- What would be an alternative analysis?

Constant Lexemes

- What kinds of words are constant lexemes in our grammar?
  Adjectives, adverbs, prepositions, determiners, conjunctions: words that do not inflect in English.

- Why do we need a rule for these words?
  They need to go 'from' *lexeme* to *word*, and only words appear in trees.

- What would be an alternative analysis?
  Put the sub-hierarchy for constant lexemes under *word* instead of under *lexeme*.

Constant Lexeme LR

$$\begin{bmatrix} \textit{i-rule} \\ \textsc{input} & \langle \boxed{1}, \textit{const-lxm} \rangle \\ \textsc{output} & \left[\textsc{first}\ \boxed{1}\right] \end{bmatrix} \quad \text{i.e.} \quad \begin{bmatrix} \textit{i-rule} \\ \textsc{input} & \langle \boxed{1}, \textit{const-lxm} \rangle \\ \textsc{output} & \langle \boxed{1}, [\,] \rangle \end{bmatrix}$$

The form does not change, and the rule says nothing else. The type *i-rule* does the work: it copies SYN and ARG-ST and makes the output a *word*.

- What keeps this from applying to, say, verb lexemes?
- Why is this an *i-rule*?

Constant Lexeme LR

$$\begin{bmatrix} \textit{i-rule} \\ \text{INPUT} \quad \langle \boxed{1}, \textit{const-lxm} \rangle \\ \text{OUTPUT} \quad \left[\text{FIRST } \boxed{1}\right] \end{bmatrix} \quad \text{i.e.} \quad \begin{bmatrix} \textit{i-rule} \\ \text{INPUT} \quad \langle \boxed{1}, \textit{const-lxm} \rangle \\ \text{OUTPUT} \quad \langle \boxed{1}, [] \rangle \end{bmatrix}$$

The form does not change, and the rule says nothing else. The type *i-rule* does the work: it copies SYN and ARG-ST and makes the output a *word*.

- What keeps this from applying to, say, verb lexemes?
  The input must be a *const-lxm*. Verb lexemes are *infl-lxm*s, a different branch of the hierarchy.

- Why is this an *i-rule*?
  It takes a *lexeme* to a *word*, which is what inflectional rules do, even though nothing is inflected.

Other Lexical Rules (So Far)

- Inflectional (*i-rule*): Singular Noun, 3rd-Singular Verb, Non-3rd-Singular Verb, Past-Tense Verb, Base Form
- Derivational (*d-rule*): Present Participle, Past Participle, Agent Nominalization
- Each one is in the grammar summary in Chapter 9. On the next slides, read each rule and ask: what is the input, what changes, and where does the rest come from?

Singular Noun Lexical Rule

$$\begin{bmatrix} \textit{i-rule} \\ \text{INPUT} \quad \langle \boxed{1} , \textit{cn-lxm} \rangle \\ \text{OUTPUT} \quad \langle \boxed{1} , \left[\text{SYN} \left[\text{HEAD} \left[\text{AGR} \left[\text{NUM } \textit{sg}\right]\right]\right]\right] \rangle \end{bmatrix}$$

Any common noun lexeme, count or mass, gives a singular word with the same form. Compare the Plural Noun LR, whose input is only *cntn-lxm*: mass nouns have no plural.

3rd-Singular Verb Lexical Rule

$$\left[\begin{array}{l} \textit{i-rule} \\ \textsc{input} \quad \left\langle \boxed{3}\ ,\ \left[\begin{array}{l}\textit{verb-lxm}\\ \textsc{sem}\ \left[\textsc{restr}\ \boxed{A}\right]\end{array}\right]\right\rangle \\ \textsc{output} \quad \left\langle \mathrm{F}_{3SG}\left(\boxed{3}\right),\ \left[\begin{array}{ll}\textsc{syn} & \left[\textsc{head}\ \left[\begin{array}{ll}\textsc{form} & \textit{fin}\\ \textsc{agr} & \textit{3sing}\end{array}\right]\right]\\ \textsc{sem} & \left[\textsc{restr}\ \boxed{A} \oplus \ldots\right]\\ \textsc{arg-st} & \left\langle\left[\textsc{case}\ \textit{nom}\right],\ \ldots\right\rangle\end{array}\right]\right\rangle \end{array}\right]$$

Add *-s*; the word is finite and 3rd singular, its subject (the first argument) is nominative, and the RESTR gets something extra (the tense).

Non-3rd-Singular Verb Lexical Rule

$$\left[\begin{array}{ll} \textit{i-rule} & \\ \text{INPUT} & \left\langle \boxed{1}\ ,\ \left[\begin{array}{ll} \textit{verb-lxm} & \\ \text{SEM} & \left[\text{RESTR}\ \boxed{A}\right] \end{array}\right] \right\rangle \\ \text{OUTPUT} & \left\langle \boxed{1}\ ,\ \left[\begin{array}{ll} \text{SYN} & \left[\text{HEAD} \left[\begin{array}{ll} \text{FORM} & \textit{fin} \\ \text{AGR} & \textit{non-3sing} \end{array}\right]\right] \\ \text{SEM} & \left[\text{RESTR}\ \boxed{A} \oplus \ldots\right] \\ \text{ARG-ST} & \left\langle \left[\text{CASE}\ \textit{nom}\right], \ldots \right\rangle \end{array}\right] \right\rangle \end{array}\right]$$

The same, but the form stays as it is and the agreement is anything but 3rd singular: *I/you/we/they walk*.

Past-Tense Verb Lexical Rule

$$\left[\begin{array}{l} \textit{i-rule} \\ \textsc{input} \quad \left\langle \boxed{3}\ ,\ \left[\begin{array}{l}\textit{verb-lxm}\\ \textsc{sem}\ \left[\textsc{restr}\ \boxed{A}\right]\end{array}\right]\right\rangle \\ \textsc{output} \quad \left\langle \mathrm{F}_{PAST}\left(\boxed{3}\right),\ \left[\begin{array}{ll}\textsc{syn} & \left[\textsc{head}\ \left[\textsc{form}\ \textit{fin}\right]\right]\\ \textsc{sem} & \left[\textsc{restr}\ \boxed{A} \oplus \ldots\right]\\ \textsc{arg-st} & \left\langle\left[\textsc{case}\ \textit{nom}\right],\ \ldots\right\rangle\end{array}\right]\right\rangle \end{array}\right]$$

Add *-ed*; finite, nominative subject, but no agreement constraint at all: *I/you/she/they walked*.

Base Form Lexical Rule

$$\begin{bmatrix} \textit{i-rule} \\ \textsc{input} & \langle \boxed{1}\ ,\ \textit{verb-lxm} \rangle \\ \textsc{output} & \langle \boxed{1}\ ,\ \left[\textsc{syn}\ \left[\textsc{head}\ \left[\textsc{form}\ \textit{base}\right]\right]\right] \rangle \end{bmatrix}$$

Same form, [FORM base]: not finite, so no nominative subject and no agreement. This is the verb in *Eat!*, *to eat* and *will eat*.

Present Participle Lexical Rule

$$\left[\begin{array}{l} \textit{d-rule} \\ \textsc{input} \quad \left\langle \boxed{3}\ ,\ \left[\begin{array}{ll} \textit{verb-lxm} & \\ \textsc{sem} & \left[\textsc{restr}\ \boxed{A}\right] \\ \textsc{arg-st} & \boxed{B} \end{array}\right] \right\rangle \\ \textsc{output} \quad \left\langle \mathrm{F}_{PRP}\left(\boxed{3}\right),\ \left[\begin{array}{ll} \textit{part-lxm} & \\ \textsc{syn} & \left[\textsc{head}\ \left[\textsc{form}\ \textit{prp}\right]\right] \\ \textsc{sem} & \left[\textsc{restr}\ \boxed{A} \oplus \ldots\right] \\ \textsc{arg-st} & \boxed{B} \end{array}\right] \right\rangle \end{array}\right]$$

A *d-rule*: ***eat*** gives the lexeme ***eating***, of type *part-lxm*, with the same arguments. *part-lxm* is a *const-lxm*, so the Constant Lexeme LR then makes it a word.

Past Participle Lexical Rule

$$\left[\begin{array}{l} \textit{d-rule} \\ \textsc{input} \quad \left\langle \boxed{3}\ ,\ \left[\begin{array}{ll} \textit{verb-lxm} & \\ \textsc{sem} & \left[\textsc{restr}\ \boxed{A}\right] \\ \textsc{arg-st} & \boxed{B} \end{array}\right] \right\rangle \\ \textsc{output} \quad \left\langle \mathrm{F}_{PSP}\left(\boxed{3}\right),\ \left[\begin{array}{ll} \textit{part-lxm} & \\ \textsc{syn} & \left[\textsc{head}\ \left[\textsc{form}\ \textit{psp}\right]\right] \\ \textsc{sem} & \left[\textsc{restr}\ \boxed{A} \oplus \ldots\right] \\ \textsc{arg-st} & \boxed{B} \end{array}\right] \right\rangle \end{array}\right]$$

The same, with [FORM psp]: ***eat*** gives ***eaten***, as in *I have eaten*.

Agent Nominalization Lexical Rule

$$\left[\begin{array}{l} \textit{d-rule} \\ \textsc{input} \quad \left\langle \boxed{2}\ ,\ \left[\begin{array}{ll} \textit{verb-lxm} & \\ \textsc{sem} & \left[\textsc{index}\ s\right] \\ \textsc{arg-st} & \left\langle \mathrm{X}_i\ ,\ \mathrm{NP}_j \right\rangle \end{array}\right] \right\rangle \\ \textsc{output} \quad \left\langle \mathrm{F}_{\textit{-er}}\left(\boxed{2}\right),\ \left[\begin{array}{ll} \textit{cntn-lxm} & \\ \textsc{sem} & \left[\textsc{index}\ i\right] \\ \textsc{arg-st} & \left\langle \mathrm{Y}\ \left(,\ \mathrm{PP}_j\left[\textsc{form}\ \textit{of}\right]\right)\right\rangle \end{array}\right] \right\rangle \end{array}\right]$$

Add *-er* to a strictly transitive verb lexeme. The result is a count noun lexeme whose index is the verb's subject index i: a ***driver*** is the one who drives. The verb's object j can only appear as an *of*-PP: *a builder of bridges*.

arg-st & ARP

- Given the ARP, what do we need to specify about the valence properties of words?
- Why isn't the ARP a constraint on the type *lexeme*?

arg-st & ARP

- Given the ARP, what do we need to specify about the valence properties of words?
  Only the length of the SPR list, and the SHAC already does that: nouns and verbs have exactly one specifier. The ARP then fills SPR and COMPS from ARG-ST.

- Why isn't the ARP a constraint on the type *lexeme*?
  Some derivational rules change the argument structure (e.g. Agent Nominalization). If SPR and COMPS were already linked to ARG-ST in the lexeme, those rules would have to change all three.

The Feature form

- Different inflected forms of verbs show up in different syntactic environments. Examples?

- These different forms are syntactically distinguished by the feature FORM, as assigned by lexical rules.

- FORM is also useful in our analyses of coordination and PP selection.

The Feature form

- Different inflected forms of verbs show up in different syntactic environments. Examples?
  I eat; *Eat!*; *I was eating*; *I have eaten*

- These different forms are syntactically distinguished by the feature FORM, as assigned by lexical rules.

- FORM is also useful in our analyses of coordination and PP selection.

What Rules These Out?

(1) **Kim eat pizza.*

(2) **Kim seems to eats pizza.*

(3) **Dana helped Leslie pack and moved.*

(4) **Kim relies for Sandy.*

(5) **Dana walked and Kim.*

What Rules These Out?

(6) **Kim eat pizza.*
eat is either Non-3rd-Singular (does not agree with Kim) or Base Form (not finite, so it cannot head a sentence)

(7) **Kim seems to eats pizza.*
Spoiler: to will be an auxiliary verb that takes a [FORM base] complement (Chapter 12), so eats does not fit

(8) **Dana helped Leslie pack and moved.*
helped selects a [FORM base] complement; the Coordination Rule makes the conjuncts and the mother share FORM, so moved cannot join pack

(9) **Kim relies for Sandy.*
relies requires a PP complement headed by a preposition with [FORM on]

(10) **Dana walked and Kim.*
Used to be fine, since the VAL values match; now the Coordination Rule requires the conjuncts to share FORM, and Kim has none

Review: Lexical Terminology

- **Lexeme**: a type in our grammar; its subtypes state generalizations about combinations of constraints
- **Lexical entry**: what we write in the lexicon; it licenses a family of lexical sequences
- **Lexical sequence**: a form paired with a fully resolved feature structure of type *lexeme* or *word*
- **Family of lexical sequences**: a form paired with an underspecified feature structure, resolvable in many ways
- **Lexical rule**: what we write to derive new lexical sequences from old ones
- **Lexical rule instantiation**: a fully resolved *l-rule*: one rule, one input, one output
- **Word structure**: the bottom of the tree: a form with a fully resolved *word* feature structure over it

Activity: Arriving at a Word Structure

Write the word structure for ***writers*** as it appears in the tree for the sentence

(11) *Those writers need inspiration.*

starting from the lexical entry for ***write***:

$$\left\langle \text{write} \ , \begin{bmatrix} \textit{stv-lxm} & & \\ \text{SEM} & \begin{bmatrix} \text{INDEX} & s & \\ \text{RESTR} & \left\langle \begin{bmatrix} \text{RELN} & \textit{write} \\ \text{SIT} & s \\ \text{WRITER} & i \\ \text{WRITTEN} & j \end{bmatrix} \right\rangle \end{bmatrix} \\ \text{ARG-ST} & \langle \text{X}_i, \ \text{Y}_j \rangle & \end{bmatrix} \right\rangle$$

Activity: Arriving at a Word Structure

Write the word structure for ***writers*** as it appears in the tree for the sentence

(12) *Those writers need inspiration.*

starting from the lexical entry for ***write***:

$$\left\langle \text{write} \ , \begin{bmatrix} \textit{stv-lxm} \\ \text{SEM} & \begin{bmatrix} \text{INDEX} & s \\ \text{RESTR} & \left\langle \begin{bmatrix} \text{RELN} & \textit{write} \\ \text{SIT} & s \\ \text{WRITER} & i \\ \text{WRITTEN} & j \end{bmatrix} \right\rangle \end{bmatrix} \\ \text{ARG-ST} & \langle \text{X}_i, \ \text{Y}_j \rangle \end{bmatrix} \right\rangle$$

Which lexical rules do we need, and in which order? Agent Nominalization LR (***write*** $\rightarrow$ ***writer***, a *d-rule*), then Plural Noun LR (***writer*** $\rightarrow$ ***writers***, an *i-rule*).

Step 1: Agent Nominalization LR

$$\left\langle \text{writer} \ , \begin{bmatrix} \textit{cntn-lxm} \\ \text{SYN} & \begin{bmatrix} \text{HEAD} & \begin{bmatrix} \textit{noun} \\ \text{AGR} \ \boxed{1} \begin{bmatrix} \text{PER} \ \textit{3rd} \end{bmatrix} \end{bmatrix} \\ \text{VAL} & \begin{bmatrix} \text{SPR} \left\langle \text{DP} \begin{bmatrix} \text{COUNT} & + \\ \text{AGR} & \boxed{1} \end{bmatrix} \right\rangle \end{bmatrix} \end{bmatrix} \\ \text{SEM} & \begin{bmatrix} \text{MODE} & \textit{ref} \\ \text{INDEX} & i \\ \text{RESTR} & \left\langle \begin{bmatrix} \text{RELN} & \textit{write} \\ \text{SIT} & s \\ \text{WRITER} & i \\ \text{WRITTEN} & j \end{bmatrix} \right\rangle \end{bmatrix} \\ \text{ARG-ST} & \left\langle \text{DP} \ (, \ \text{PP}_j \left[\text{FORM} \ \textit{of} \right]) \right\rangle \end{bmatrix} \right\rangle$$

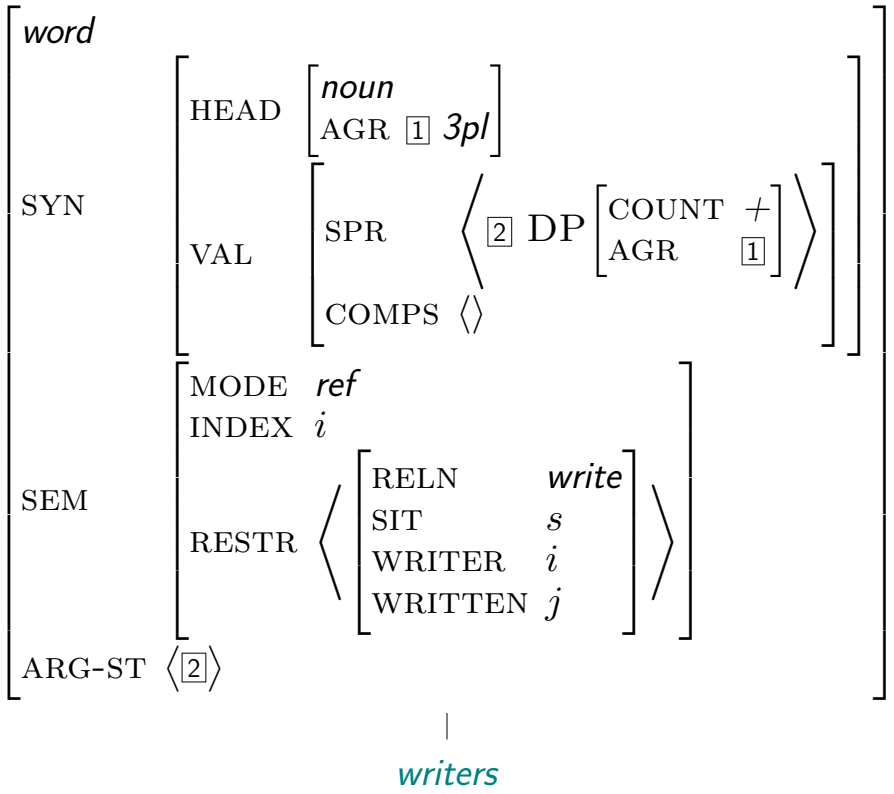
The rule adds *-er*, changes the type to *cntn-lxm* and makes the index the writer, i; the RESTR is unchanged. The noun types add the rest. The verb's object j survives only as an optional *of*-PP.

Step 2: Plural Noun LR

$$\left\langle \text{writers} \ , \begin{bmatrix} \textit{word} \\ \text{SYN} & \begin{bmatrix} \text{HEAD} & \begin{bmatrix} \textit{noun} \\ \text{AGR} \ \boxed{1} \ \textit{3pl} \end{bmatrix} \\ \text{VAL} & \begin{bmatrix} \text{SPR} & \boxed{\text{B}} \left\langle \text{DP} \begin{bmatrix} \text{COUNT} & + \\ \text{AGR} & \boxed{1} \end{bmatrix} \right\rangle \\ \text{COMPS} & \boxed{\text{C}} \ \langle\rangle \end{bmatrix} \end{bmatrix} \\ \text{SEM} & \begin{bmatrix} \text{MODE} & \textit{ref} \\ \text{INDEX} & i \\ \text{RESTR} & \left\langle \begin{bmatrix} \text{RELN} & \textit{write} \\ \text{SIT} & s \\ \text{WRITER} & i \\ \text{WRITTEN} & j \end{bmatrix} \right\rangle \end{bmatrix} \\ \text{ARG-ST} & \boxed{\text{B}} \oplus \boxed{\text{C}} \end{bmatrix} \right\rangle$$

Add *-s* and make it a *word* with [NUM pl]. Now the ARP applies: SPR is the DP and COMPS is empty, since here ***writers*** has no *of*-PP.

Activity: The Word Structure for *writers*



The form *writers* under a feature structure of type *word*, licensed by two lexical rules from the entry for ***write***.

Overview

- How lexical rules fit in
- Three types of lexical rules, constraints
- Example: Plural noun lexical rule
- Advice on writing lexical rules
- Constant lexemes and the other lexical rules (so far)
- ARG-ST & ARP; the feature FORM
- Activity: the word structure for ***writers***

Problem

P1: Arguments in Japanese

As noted in Chapter 2, Japanese word order differs from English in a number of ways, including the fact that it is a 'Subject-Object-Verb' (SOV) language. The next two slides give some relevant examples. In the glosses, 'NOM', 'ACC', and 'DAT' stand for nominative, accusative, and dative case, respectively. (Note that Japanese has one more case — dative — than English does. This doesn't have any important effects on the analysis; it merely requires that we posit one more possible value of CASE for Japanese than for English.)[1]

[1] The examples marked with '*' here are unacceptable with the indicated meanings. Some of these might be well-formed with some other meaning of no direct relevance; others might be well-formed with special intonation that we will ignore for present purposes.

P1: Arguments in Japanese — Data

(13) *Hitorino otoko-ga sono hon-o yonda.*
one man-NOM that book-ACC read.PST
'One man read that book.'
[cf. *Yonda hitorino otoko-ga sono hon-o.
*Hitorino otoko-ga yonda sono hon-o.
*Otoko-ga hitorino sono hon-o yonda.
*Hitorino otoko-ga hon-o sono yonda.
*Hitorino otoko-ni/-o sono hon-o yonda.
*Hitorino otoko-ga sono hon-ga/-ni yonda.]

P1: Arguments in Japanese — Data (continued)

(14) *Hanako-ga hon-o yonda.*
Hanako-NOM book-ACC read.PST
'Hanako read the book(s).'
[cf. *Yonda Hanako-ga hon-o.
*Hanako-ga yonda hon-o.
*Hanako-ni/-o hon-o yonda.
*Hanako-ga hon-ni/-ga yonda.]

P1: Arguments in Japanese — More Data

(15) *Sensei-ga Taroo-ni sono hon-o ageta.*
teacher-NOM Taroo-DAT that book-ACC give.PST
'The teacher(s) gave that book to Taroo.'
[cf. *Ageta sensei-ga Taroo-ni sono hon-o.
*Sensei-ga ageta Taroo-ni sono hon-o.
*Sensei-ga Taroo-ni ageta sono hon-o.
*Sensei-o/-ni Taroo-ni sono hon-o ageta.
*Sensei-ga Taroo-ga/-o sono hon-o ageta.
*Sensei-ga Taroo-ni sono hon-ga/-ni ageta.]

(16) *Hanako-ga kita.*
Hanako-NOM arrive.PST
'Hanako arrived.'
[cf. *Kita Hanako-ga.]

P1: Arguments in Japanese — Tasks

As the contrasting ungrammatical examples show, the verb must appear in final position in Japanese. In addition, we see that verbs select for NPs of a particular case, much as in English. In the following tasks, assume that the nouns and verbs of Japanese are inflected words, derived by lexical rule from the appropriate lexemes.

A. Write Head-Specifier and Head-Complement Rules for Japanese that account for the data illustrated here. How are they different (if at all) from the Head-Specifier and Head-Complement Rules for English?

P1: Arguments in Japanese — Tasks (continued)

B. Give the lexical entry for each of the verbs illustrated in (13)–(16).

   [*Make sure your entries interact with the rules you formulated in part (A) to account for the above data. The data given permit you to specify only some features; leave others unspecified. Assume that there is a Past-Tense Verb Lexical Rule (an i-rule) that relates your lexical entries to the words shown in (13)–(16). We have not provided a hierarchy of lexeme types for Japanese. You may either give all relevant constraints directly on the lexical entries, or posit and use subtypes of lexeme. In the latter case, you must also provide those types.*]

C. Give the lexical entries for the nouns *Taroo* and *hon*. [*See notes on part (B).*]

D. Formulate the lexical rule for deriving the inflected forms ending in *-o* from the nominal lexemes.

Acknowledgments and References

- Course design and slides borrow heavily from Emily Bender's course: *Linguistics 566: Introduction to Syntax for Computational Linguistics* http://courses.washington.edu/ling566

- The slides on lexical types and lexical rules are adapted from Emily Bender's, with her permission; the feature structures follow Sag et al. (2003), © 2003 CSLI Publications

- This lecture complements Sag et al. (2003, Chapter 8)

*

References

Ivan A. Sag, Tom Wasow, and Emily Bender. 2003. *Syntactic Theory: A Formal Introduction*. CSLI Publications, Stanford, second edition.